

Time-Domain Synthesis of Gravitational-Wave Detector Glitches using Class-Conditional Derivative Generative Adversarial Networks

Tom Dooney^{1,2}, Mees de Boer², Harsh Narola^{1,2}, Melissa Lopez^{1,2},
Stefano Bromuri³, Daniel Stanley Tan³, and Chris Van Den Broeck^{1,2}

¹*Nikhef, Science Park 105, 1098 XG Amsterdam, The Netherlands*

²*Institute for Gravitational and Subatomic Physics (GRASP),*

Utrecht University, Princetonplein 1, 3584 CC Utrecht, The Netherlands and

³*Faculty of Science, Open Universiteit, Valkenburgerweg 177, 6419 AT Heerlen, The Netherlands*

(Dated: August 19, 2026)

Gravitational-wave detectors such as LIGO, Virgo, and KAGRA are highly sensitive instruments susceptible to numerous noise sources. Short-duration transient noise events, known as glitches, pose a particular challenge for data analysis pipelines as they can mimic or obscure astrophysical signals. We present GlitchGAN, a class-conditional generative model, built upon the Conditional Derivative GAN (cDVGAN) architecture, that is capable of synthesizing glitches directly in the time domain. The model is trained on high-quality reconstructions of seven common glitch types observed during LIGO’s third observing run (O3): Blip, Fast Scattering, Koi Fish, Low-Frequency Burst, Scattered Light, Tomte, and Whistle. We show that GlitchGAN generalizes effectively, learning to reproduce a diverse glitch space that is consistent with these reconstructions. Moreover, because the model is conditioned on glitch class, it can generate *hybrid* or transitional glitch morphologies by interpolating across the class-conditioning vector after training. GlitchGAN generates 1000 glitches in under 22 seconds on a CPU, making it suitable for large-scale glitch synthesis for detector simulations, mock data challenges, and pipeline validation. Synthetic glitches are validated against real glitch reconstructions using the Gravity Spy classifier, widely used in the GW community for glitch classification, and an unsupervised analysis using UMAP embeddings. Gravity Spy classifies the majority of GlitchGAN’s synthetic glitches as the correct class while the UMAP analysis shows substantial overlap between real and synthetic samples in the reduced latent space. We further validate synthetic glitches with a downstream convolutional neural network trained to distinguish real glitches held out from GlitchGAN’s training from synthetic glitches produced by the model. While this classifier reliably detects synthetic samples in a clean, noise-free representation, this detectability drops substantially once real and synthetic glitches are injected into realistic detector noise, indicating that the residual statistical differences are subtler than they first appear. Despite remaining statistically distinguishable, we show that GlitchGAN-generated glitches are practically useful: augmenting a severely data-scarce real training set with synthetic samples improves downstream multi-class glitch classification accuracy, outperforming simple duplication of the same limited real data. We further highlight a critical limitation of magnitude-only spectrograms: classifiers operating on magnitude Q -transforms can confidently misclassify physically unrealistic glitches from less robust models, underscoring the need for complementary validation methods that preserve phase information.

I. INTRODUCTION

The detection of gravitational waves (GWs) by the LIGO [1] and Virgo [2] detectors has opened a new observational window on the universe, enabling measurements of compact binary coalescences and other cataclysmic astrophysical events [3]. The extreme sensitivity required for these detections, however, renders the instruments susceptible to instrumental and environmental noise sources [4]. Among these, short-duration transient noise artefacts known as *glitches* [5–7] represent a persistent challenge for data analysis pipelines and extensive work is required to mitigate their impact, ensuring the quality of gravitational-wave data [8]. Glitches are unmodelled noise events that exhibit diverse morphologies—ranging from impulsive millisecond-duration events to extended, dispersive features spanning several seconds—and can mimic or obscure astrophysical signals [9–11]. When a glitch overlaps with a GW signal it can significantly bias parameter estimation [12–14],

and approximately 25 of the ~ 90 confident detections up to the end of O3 required some form of glitch mitigation [3, 12, 15–17]. The contamination of astrophysical signals by glitches will be exacerbated in next-generation detectors. Einstein Telescope [18] and Cosmic Explorer [19] are expected to detect on the order of 10^4 – 10^5 binary black hole and neutron star mergers per year [20, 21], thus giving rise to more frequent glitch-signal overlaps.

The ability to synthesize realistic glitch populations is therefore of growing importance. Such capability enhances the realism of detector simulations and supports software injection campaigns [6, 22, 23], enabling more rigorous validation of GW detection pipelines and parameter estimation algorithms. Realistic synthetic glitches are also essential for the construction of mock data challenges (MDCs) and for data augmentation in machine learning-based glitch classification systems. A possible approach is to model glitches mathematically or using machine learning, given *a priori* knowledge of their production mechanism [24, 25]. Other works have developed

models that mimic their principal characteristics using a few parameters [26]. However, such approaches either require prior knowledge of the glitch’s origin, which isn’t always available, or are limited to a restricted parametric form, making them difficult to scale across the full diversity of glitch morphologies.

Recently, generative models have been applied broadly within GW data analysis to address multiple challenges [27, 28]. For glitch synthesis, Generative Adversarial Networks (GANs) [29] have been used to augment spectrogram-based datasets for downstream classification tasks [30, 31]. The *gengli* glitch generator [32, 33] demonstrated that a GAN trained on Blip glitches extracted from detector data using BayesWave [34], the state-of-the-art method for reconstructing glitches from detector data, can successfully learn their underlying time-domain distribution, providing a useful framework for simulations and MDCs. McGANn [35] implemented a conditional GAN capable of synthesizing five distinct waveform classes analogous to GW bursts in one model, with class interpolation enabling the generation of hybrid morphologies that blend features from the learned classes. Powell et al. [31] implemented generative modelling on a broader glitch class space by training separate GANs on magnitude Q -scan representations of each glitch class. However, extending this approach to a single model operating directly in the time domain remains unexplored, despite its advantages in preserving phase information and enabling direct use of time-series data in simulations and MDCs.

The principal obstacle to realistic time-domain glitch synthesis has been the availability of high-quality training data. BayesWave [34] is computationally expensive, and converges unreliably for certain glitch morphologies, making it impractical to construct the large-scale, diverse datasets that cover the broad glitch space. The recent development of DeepExtractor [36]—a deep learning framework for time-domain glitch reconstruction—offers fast, high-fidelity reconstructions that preserve both amplitude and phase information without imposing waveform smoothness assumptions. DeepExtractor has been shown to yield better time-domain mismatch performance than BayesWave in a simulated glitch recovery experiment for both in-sample training classes and out-of-sample classes unseen during training (see FIG. 7 in [36]), achieving a $10,000\times$ speed-up over BayesWave in realistic 1:1 comparisons.

Furthermore, DeepExtractor has been shown to be able to reconstruct three real signals in both LIGO’s Hanford and Livingston detectors during O3 with high fidelity without being trained on gravitational waveforms. DeepExtractor achieved low time-domain mismatch against the maximum-likelihood templates from standard parameter estimation workflows (see FIG. 11 in [36]), where all single-detector signals with matched-filter signal-to-noise ratio (SNR) $\rho_{MF} > 13.5$ are reconstructed with under 10% mismatch against maximum-likelihood templates. Such findings support DeepExtrac-

tor as a reliable tool for glitch reconstruction that is agnostic of the underlying morphology, making it suitable for generating large-scale training datasets for generative models.

In this paper, we introduce GlitchGAN, a class-conditional model trained on a dataset of DeepExtractor reconstructions of seven common glitch classes observed during LIGO’s O3 run: Blip, Fast Scattering, Koi Fish, Low-Frequency Burst, Scattered Light, Tomte, and Whistle. Examples of the glitches used for training and the corresponding GlitchGAN outputs are shown in Figure 1. GlitchGAN inherits the Conditional Derivative GAN (cDVGAN) architecture [37, 38], which features an auxiliary discriminator applied to the time derivative of signals, providing additional regularization that improves training stability and sample fidelity compared to standard GAN counterparts.

We validate the quality of the DeepExtractor training data using the Gravity Spy classifier [39–41] and unsupervised t-SNE [42] and UMAP embeddings [43], which have been used in multiple studies to characterize the morphological distribution of glitches in LIGO data [44–46]. The evaluation establishes a realistic baseline against which synthetic glitches can be assessed. Applying the same evaluation to GlitchGAN’s outputs, we show that the model captures much of the morphological diversity across all seven classes, with substantial overlap between real and generated distributions in the t-SNE and UMAP latent spaces and with the majority of synthetic glitches correctly identified by Gravity Spy.

Furthermore, we train a dedicated CNN binary classifier to distinguish real glitches held out from training from GlitchGAN-generated samples. While this classifier reliably detects synthetic samples in the clean, noise-free representation used for training (95.7% accuracy), a simple linear logistic regression baseline classifier performs substantially worse (77.6% accuracy), indicating that the residual difference is largely nonlinear rather than a significant morphological mismatch. Injecting both populations into realistic detector noise reduces this detectability further (66.0% CNN accuracy), and we show that despite remaining statistically distinguishable, GlitchGAN’s synthetic glitches are practically useful: augmenting a severely data-scarce real training set with synthetic samples substantially improves downstream multi-class classification accuracy over both the unaugmented baseline and naive duplication of the same limited real data.

We also demonstrate that GlitchGAN can interpolate between classes, producing novel hybrid morphologies that mix features from the learned classes. Finally, we highlight a fundamental limitation of magnitude-only time-frequency representations as validation tools: when applied to physically unrealistic glitches from a diffusion-based comparison model [47], Gravity Spy assigns correct class labels with high confidence despite clear inconsistency in the time domain, because the Q -transform discards phase information entirely. This underscores the necessity of complementary, phase-preserving validation

methods.

The remainder of the paper is organized as follows: Section II describes the preparation of our training dataset. Section III details the GlitchGAN architecture, training procedure and evaluation methodology. Section IV presents results across multiple metrics. Finally, Section V discusses our findings and opportunities for future work.

II. DATA

A. Data curation

We construct our training dataset using DeepExtractor reconstructions of real LIGO glitches. To ensure high-quality, morphologically representative samples, we apply strict selection criteria:

1. Gravity Spy classification confidence $\sigma_{\text{GS}} \geq 0.9$, ensuring that the reconstructed glitches are representative of well-characterized classes.
2. Signal-to-noise ratio (SNR) ≥ 15 (sufficient amplitude to minimize reconstruction artifacts)
3. Detectors: LIGO Hanford (H1) and Livingston (L1)
4. Observing runs: O3a and O3b

We include seven glitch classes in the training of GlitchGAN, namely; Blip, Fast Scattering, Koi Fish, Low-Frequency Burst, Scattered Light, Tomte, and Whistle.

Blip glitches have a characteristic morphology of a symmetric teardrop shape in time-frequency in the range [30, 250] Hz with short-durations, ~ 0.04 s [11]. They appear in both Hanford and Livingston, as well as Virgo and GEO 600 [48]. Due to their abundance and form, these glitches hinder both the unmodeled burst and modelled CBC searches, with particular emphasis on compact binaries with large total mass, highly asymmetric component masses, and spins anti-aligned with the orbital angular momentum [9].

Fast Scattering glitches appear as short-duration arches ($\sim 0.2 - 0.3$ s) in the frequency range [20, 60] Hz. These glitches are strongly correlated with ground motion in range [0.1 – 0.3] Hz and [1 – 6] Hz, which in turn is associated with thunderstorms and human activity near the detector. Note that this class was added during O3, and it is more abundant in Livingston than Hanford, due to differences in ground motion and detector sensitivity [49].

Koi Fish glitches get their name from their frontal resemblance to Koi Fishes. They are also similar to *Blips* but typically feature a higher SNR, spanning the frequency range of [20, 1000] Hz.

Low-Frequency Bursts are short-duration glitches (~ 0.25 s) in the frequency range [10 – 20] Hz with a distinctive blob shape. These occurrences were prevalent in Livingston data during O1 and Hanford data in O3a.

Scattered Light glitches have longer duration harmonics ($\sim 2.0 - 4.0$ s), and in the time-frequency domain, they appear as arches being often stacked on top of each other. These glitches are problematic since their frequency content lies in the band of interest of GW astrophysical events. In O3, they were found to be coupled with the relative motion between the optical suspension system’s end test-mass chain and the reaction-mass chain [49].

Tomte glitches are also short-duration (~ 0.25 s) with characteristic triangular morphology.

Whistles have a characteristic V, U or W shape at higher frequencies with typical durations ~ 0.25 s. They are caused when radio-frequency signals beat with the voltage-controlled oscillators [50].

These seven classes collectively span wide time and frequency ranges, encompassing much of the morphological variability relevant to transient instrumental noise. While it is a common glitch class, we exclude the Extremely Loud class because its characterization is dominated by exceptionally high SNR rather than distinctive morphology, making it a poor target for generative modeling.

B. Dataset Preprocessing

We reconstruct with DeepExtractor a 2-second window centered on the glitch trigger time at a sampling rate of 4,096 Hz, yielding 8,192 samples per glitch waveform. Our goal was to obtain approximately 5,000 samples per class (1,250 per detector per observing run) under the selection criteria above. Unfortunately, data availability constraints limited acquisition for some classes. Fast Scattering yielded only 2,774 samples, and Low-Frequency Burst provided 4,316 samples. The total curated dataset contained 31,511 samples.

To mitigate training instabilities from class imbalance, we applied bootstrap resampling (sampling with replacement) to oversample all classes to 5,000 samples each, yielding a balanced training set of 35,000 total samples. Dataset composition by detector, observing run, and class is detailed in Table I. Before training, all reconstructed time series are normalized to the range $[-1, 1]$, followed by subtracting the mean. This normalization constrains the generative model to operate at a single effective SNR scale, but generated samples can be rescaled post-generation to any desired SNR for downstream applications.

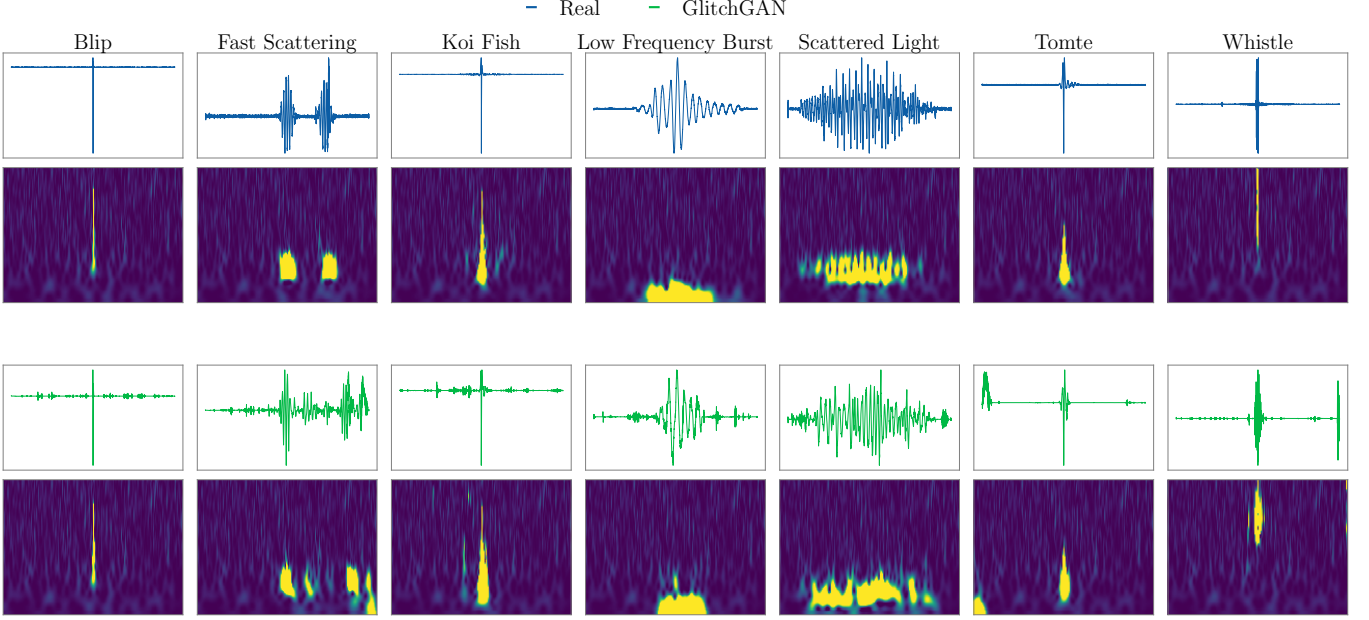


FIG. 1: Comparison of real and GlitchGAN-generated glitches across the seven glitch classes used for training, shown in both representations. **Row 1 (Real, time domain):** DeepExtractor reconstructions of real glitches. **Row 2 (Real, Q-transform):** Corresponding Q-scans of the same real glitches. **Row 3 (Synthetic, time domain):** Glitches generated by GlitchGAN. **Row 4 (Synthetic, Q-transform):** Corresponding Q-scans of the generated glitches. Each column shows one representative sample per class.

TABLE I: Composition of the curated glitch dataset. Number of samples per class, detector (H1: Hanford, L1: Livingston), and observing run (O3a, O3b). Classes are bootstrapped to 5000 samples each for balanced training.

Glitch Class	H1		L1	
	O3a	O3b	O3a	O3b
Blip	1246	1247	1245	1243
Fast Scattering	143	143	1244	1244
Koi Fish	1247	1248	1243	1247
Low Freq. Burst	1246	1246	912	912
Scattered Light	1236	1243	1246	1248
Tomte	1031	1031	1247	1246
Whistle	1232	1232	1231	1232

III. METHODS

A. Model architecture: cDVGAN

Generative adversarial networks (GANs) [29], wherein a generator and discriminator are trained in an adversarial min-max game, have established themselves as a powerful framework for learning complex high-dimensional data distributions. Wasserstein GANs (WGANs) [51] improve training stability by replacing the Jensen–Shannon divergence with the Wasserstein-1 distance, and the gradient penalty variant (WGAN-GP) [52] further stabilizes optimization by enforcing a Lipschitz constraint on the discriminator. Conditional extensions [53] allow the

generator to be steered towards a target class, enabling multi-class synthesis within a single model.

cDVGAN [37] is a conditional WGAN-GP that augments the standard two-player adversarial game with an auxiliary discriminator applied to the first-order time derivative of signals. Here we summarise the key components; full layer-by-layer architecture details are shown in Appendix A.

1. Training objective

Under the WGAN-GP framework, the loss for discriminator D is

$$L_D = \mathbb{E}_{\hat{x}}[D(\hat{x}, c)] - \mathbb{E}_x[D(x, c)] + \lambda \mathbb{E}_{\tilde{x}} \left[(\|\nabla_{\tilde{x}} D(\tilde{x}, c)\|_2 - 1)^2 \right], \quad (1)$$

where x and $\hat{x}$ are real and generated samples respectively, $\tilde{x}$ is a random interpolation between them, and $\lambda = 10$ is the gradient-penalty weight. The generator is updated to minimize

$$L_G = -\mathbb{E}_{\hat{x}}[D(\hat{x}, c)]. \quad (2)$$

In cDVGAN a second discriminator D_∂ operates on the first-order derivative $\dot{x} = dx/dt$. The combined generator loss is

$$L_G^{\text{cDVGAN}} = -\frac{1}{2} \left(\mathbb{E}_{\hat{x}}[D(\hat{x}, c)] + \mathbb{E}_{\hat{x}}[D_\partial(\dot{\hat{x}}, c)] \right), \quad (3)$$

where both discriminators contribute equally. Each discriminator is updated five times per generator update.

2. Generator

The generator G maps a noise vector $z \sim \mathcal{N}(0, I_{100})$ and a one-hot class vector $c \in \mathbb{R}^7$ to a time series of length 8,192. The class vector is first projected to a 32-dimensional embedding and concatenated with z , then passed through a dense layer and reshaped before five successive upsample-and-convolve blocks with batch normalization and ReLU activations expand the sequence to the target length.

3. Discriminators

Both D and D_∂ are 1D convolutional networks that map an input sequence and a class vector to a scalar critic value. Class conditioning uses projection [53]: a learned class embedding is dot-producted with the discriminator’s feature vector and added to the scalar output. D operates directly on the 8192-sample waveform; D_∂ operates on the 8191-sample first-order difference sequence $\dot{x} = x_{t+1} - x_t$. The derivative discriminator provides additional regularization by penalizing unphysical high-frequency artefacts in generated waveforms [38].

B. Training procedure

The model was trained for 500 epochs with a batch size of 64 using the RMSprop optimizer and a learning rate of 10^{-4} for all network components. Training was performed on a NVIDIA A100 GPU (approximately 12 hours). Although training was conducted for the full 500 epochs, the final GlitchGAN configuration uses the model weights saved at epoch 210.

The selection of epoch 210 was based on a combination of qualitative and quantitative assessments. Generated glitches were inspected throughout training, and epoch 210 was found to produce the most realistic glitch morphologies. This choice is further supported by the generator loss evolution (Appendix B), which exhibits relatively stable behaviour around this epoch compared to other stages of training. Additionally, the UMAP analysis presented in Section IV B 2 shows improved overlap between the distributions of real and generated glitches at epoch 210 relative to models saved at epochs 100, 200, 300, 400, and 500.

No hyperparameter tuning was performed for this dataset; the architecture and hyperparameters are similar to those validated in [37], outside of the increased dimensionality of the class vector, allowing us to assess out-of-the-box generalization to this more complex and realistic glitch population. The implementation uses TensorFlow [54] and is publicly available at <https://github.com/tomdooney95/glitchgan>.

C. Comparison with a diffusion-based model

As a comparison against our cDVGAN architecture in GlitchGAN, we also trained another type of generative model on our dataset known as a latent Diffusion Transformer (DiT), which was adapted for 1D time series. Diffusion models [47, 55, 56] learn to generate data by explicitly modelling the process of iteratively adding and then removing noise from samples drawn from a simple prior distribution. They have become increasingly popular in recent years for their ability to generate realistic synthetic data. Full-resolution diffusion proved computationally prohibitive, so we first trained a convolutional autoencoder to compress inputs from (8192, 1) to a latent space of (256, 16), reducing computational cost by approximately three orders of magnitude while preserving essential morphological structure. The DiT was then trained in this latent space, with generated samples decoded back to the time domain for evaluation. This model serves primarily to illustrate the limitation of magnitude-only validation metrics, as discussed in Section IV.

D. Evaluation Methodology

We evaluate both the training data (DeepExtractor reconstructions) and GlitchGAN-generated glitches using two complementary approaches: classification with the Gravity Spy classifier and visualization of low-dimensional embeddings via t-SNE and UMAP. For DeepExtractor, this establishes a quantitative baseline confirming the training data is morphologically representative; for GlitchGAN, it assesses how faithfully the model reproduces the underlying glitch distributions. We further introduce a downstream binary classification test, in which a dedicated convolutional network attempts to distinguish held-out real glitches from GlitchGAN-generated ones, and repeat it with both real and generated glitches injected into realistic detector noise at per-sample SNRs drawn from the empirical distribution of each class. Finally, we evaluate a practical downstream application: whether, under the same realistic noise conditions, GlitchGAN-generated glitches can augment a severely data-scarce real training set to improve seven-class glitch classification accuracy.

1. Gravity Spy Classification

We reconstruct with DeepExtractor 100 randomly selected glitches from the Gravity Spy dataset according to the criteria in Section II, drawing 25 samples per detector (H1, L1) per observing run (O3a, O3b). Since Gravity Spy classifies glitches surrounded by detector background noise i.e. $g(t) + n(t)$, each DeepExtractor reconstruction $\hat{g}(t)$ is injected into a segment of ‘quiet’

Hanford detector noise from O3 between 1262540000 and 1262540040 GPS time prior to reclassification (representing $n(t)$), using the `fetch_open_data` method from the package `gwpy` [57]. This stretch of data was visually inspected using Q -scans to confirm the absence of glitches or other significant noise features.

We follow a similar approach to evaluate the GlitchGAN-generated synthetic glitches, injecting them into the same noise segments before classification. The only difference is that we have to explicitly rescale the generated glitches to a fixed SNR to ensure they are sufficiently loud for classification, since the generator is trained on normalized waveforms and does not learn an explicit SNR distribution. We fix the SNR of each glitch class according to the mean SNR of the respective class considering the selection criteria described in Section II. The mean SNR values used for each class are shown in Table II.

Since each per-class accuracy estimate is derived from only $n = 100$ trials, we report 95% confidence intervals alongside all classification accuracies (for both DeepExtractor and GlitchGAN) to quantify sampling uncertainty. Because per-class accuracy is a binomial proportion (the fraction of n trials classified into the correct class), we compute intervals using the Wilson score method [58], which remains bounded within $[0, 1]$ and maintains accurate coverage even for small sample sizes or accuracies near the extremes (e.g. 100%, as with the Low Frequency Burst class). For the mean Gravity Spy confidence scores, which are continuous and often skewed toward high values rather than normally distributed, we instead report 95% confidence intervals obtained via a percentile bootstrap: resampling the observed confidence values with replacement 10,000 times and taking the 2.5th and 97.5th percentiles of the resulting distribution of means.

TABLE II: Mean SNR per glitch class used for injections in the GlitchGAN evaluation.

Glitch Class	Mean SNR
Blip	29.8
Fast Scattering	36.4
Koi Fish	187.2
Low Frequency Burst	40.3
Scattered Light	31.5
Tomte	25.4
Whistle	27.1

2. Dimensionality Reduction

To visualize and compare the high-dimensional feature space of real and synthetic glitches, we use two complementary nonlinear dimensionality reduction methods: *t-distributed Stochastic Neighbor Embedding* (t-SNE) [42] and *Uniform Manifold Approximation and Projection*

(UMAP) [43]. While *Principal Component Analysis* (PCA) captures global variance through linear projections, it cannot resolve the nonlinear structure present in morphologically diverse glitch populations; t-SNE and UMAP are instead designed to preserve local cluster structure in the embedding. t-SNE embeds data by minimizing the Kullback–Leibler divergence between probability distributions encoding pairwise similarities in the original and reduced spaces; it excels at revealing local cluster structure but can distort global geometry. UMAP constructs a nearest-neighbour graph in the high-dimensional space and optimises a low-dimensional counterpart to preserve both local and global topological structure, scaling efficiently to the dataset sizes used here.

For DeepExtractor’s reconstructions, we aim to verify that the reconstructed glitches cluster according to their correct Gravity Spy class in an unsupervised manner, confirming that the reconstructions preserve the morphological characteristics recognized by Gravity Spy. For GlitchGAN-generated synthetic glitches, we evaluate whether they cluster with the real reconstructions of their respective classes (using the same UMAP embedding for real and synthetic samples), indicating that the model has successfully learned to reproduce the underlying morphological distribution of each class. The hyperparameters and seeds used for t-SNE and UMAP analyses are detailed in Appendix C.

3. Downstream Classification of real vs. synthetic glitches

To evaluate whether GlitchGAN-generated glitches are distinguishable from real DeepExtractor reconstructions, we train a binary CNN classifier to discriminate between the two populations. To this end, we train a new GlitchGAN model on the same dataset, but holding out 10% of the data for the downstream classification, stratifying the split by class. This amounts to 28,360 real training samples across the seven glitch classes, which we rebalance to 5,000 samples per class (35,000 total) following the same procedure used for the main model, and 3,151 held-out real samples that are never seen during training of this holdout GlitchGAN. This holdout model is trained separately from, and does not replace, the main GlitchGAN model reported throughout this work.

Once training of the holdout model is complete, we use its generator to produce one synthetic glitch for each of the 3,151 held-out real glitches, conditioned on the same class label, giving a balanced evaluation pool of 6,302 samples (3,151 real, 3,151 generated). We evaluate using the holdout generator’s checkpoint at epoch 210, similar to the main GlitchGAN model rather than the final epoch of training.

We use a 1D CNN with three convolutional layers followed by two fully-connected layers to perform the classification, deliberately kept simpler than and architecturally independent of GlitchGAN’s own discrimina-

tor, so that it constitutes a genuinely independent check rather than a re-run of the adversarial training objective. The full architecture is given in Appendix D.

The balanced real/generated pool is split 70/10/20 into training, validation, and test sets, stratified jointly by class and by real/fake domain – giving 4,412 training samples, 632 validation samples, and 1,258 test samples, with real and generated samples represented in equal number within every class and every split (Table III). Prior to classification, both real and generated samples are scaled identically via the same per-sample min-max and mean-subtraction procedure used to prepare GlitchGAN’s training data, so the classifier cannot exploit a trivial normalization mismatch between the two domains. As an additional check on the strength of the classification signal, we also fit a logistic regression baseline model directly on the raw waveform, with its L2 regularization strength selected via the validation split, to establish how much of any discriminating information is accessible to a model with no capacity for nonlinear feature extraction. We report test accuracy together with its 95% Wilson score confidence interval, computed using the same procedure described in Section III D 1, for both classifiers.

Class	Real	Generated	Train	Val	Test
Blip	498	498	698	100	198
Fast Scattering	277	277	388	56	110
Koi Fish	498	498	698	100	198
Low Frequency Burst	432	432	604	86	174
Scattered Light	497	497	696	100	198
Tomte	456	456	638	92	182
Whistle	493	493	690	98	198
Total	3151	3151	4412	632	1258

TABLE III: *Per-class breakdown of the held-out real and GlitchGAN-generated glitches used in the downstream real-vs-generated classification test. The 6,302-sample pool (3,151 real, 3,151 generated) is split 70/10/20 into training, validation, and test sets, stratified jointly by class and by real/fake domain, so real and generated samples are represented in equal number within every class and every split.*

Having established this experiment in a clean, noise-free representation, we additionally repeat the experiment with real and generated glitches injected into simulated Gaussian detector noise, more closely resembling how these samples would be encountered in practice.

We draw a coloured noise realization from the LIGO design power spectral density (PSD) $S_n(f)$ and whiten it. A new, independent noise realization is drawn for every injected sample, rather than reusing a single shared segment, so that no particular realization’s random fluctuations can act as a feature shared across the whole dataset.

Each glitch, real or generated, is independently rescaled to a target signal-to-noise ratio ρ before injection

according to

$$\rho = \|h\|_2 = \left(\sum_t h(t)^2 \right)^{1/2}, \quad (4)$$

where $h(t)$ is the whitened-frame glitch waveform; because our whitening procedure gives noise with unit variance per time sample, the optimal matched-filter SNR reduces exactly to this time-domain ℓ_2 norm.

Rather than injecting every sample of a given class at that class’s single, fixed mean SNR (as in Section III D 1), each individual sample here is assigned its own target SNR, drawn independently from the empirical distribution of real per-class SNR values. We construct, for each class, an empirical pool of observed SNR values from the Gravity Spy glitch catalog, applying the same selection criteria used to construct the training dataset (Section II). The resulting empirical SNR distributions for all seven classes are shown in Appendix E. For each injected sample, a target SNR is drawn with replacement from its class’s empirical pool; both real and synthetic domains draw from the identical per-class distribution, so SNR itself cannot reveal a sample’s origin to a downstream classifier.

4. Multi-class classification and data augmentation

As a practical downstream application, we evaluate whether GlitchGAN-generated glitches are useful for augmenting a real training set in a seven-class glitch classification task, particularly when real data is scarce, injecting real and generated glitches into realistic detector noise following the same procedure as Section III D 3.

We use the same held-out real glitch set as in Section III D 3 (3,151 samples across seven classes, never seen during training of the holdout GlitchGAN), and a seven-class softmax variant of the binary classifier described above: identical convolutional backbone (see Section D), but with a final seven-unit softmax layer and a sparse categorical cross-entropy loss in place of the single sigmoid output.

To evaluate the effect of real-data scarcity on the value of synthetic augmentation, we construct training pools at three scarcity levels, all stratified proportionally by class from the held-out real set, using train/validation/test splits of 70/10/20, 10/20/70, and 3/20/77 respectively: an abundant pool (the full held-out set at its natural split, giving 2,206 real training samples total, ~ 194 –349 per class), a moderately scarce pool ($\sim 10\%$ train fraction, giving 316 real training samples total, ranging from 28 to 50 per class), and a severely scarce pool ($\sim 3\%$ train fraction, giving 95 real training samples total, ranging from 8 for Fast Scattering to 15 for the larger classes). In each case, the validation and test sets are drawn from the same held-out real set at the stated split fractions and used identically across every training configuration described below for direct comparability.

For each scarcity level, we compare four training-set compositions at two augmentation-target scales – each pool’s own majority-class count, and an aggressive target of 5,000 samples per class – all injected into realistic detector noise and evaluated against the same held-out, noise-injected real validation and test sets:

1. **Unaugmented:** the real unbalanced training pool as-is.
2. **Bootstrap-duplicated:** the same real training pool, balanced via oversampling with replacement to the target scale; a control that isolates whether simply having a larger, but entirely non-novel, training set explains any observed improvement.
3. **Augmented:** the real training pool, balanced via supplementation with GlitchGAN-generated samples (conditioned on the same class label) to reach the same target scale.
4. **Synthetic-only:** GlitchGAN-generated samples only, at the target scale, with no real training data at all.

IV. RESULTS

A. Validation of DeepExtractor training data

1. Classification evaluation using Gravity Spy

Table IV summarizes the results of classifying 700 DeepExtractor reconstructions (100 per class) with Gravity Spy. Of the 700 reconstructed glitches, 627 are correctly identified by Gravity Spy, corresponding to 89.6% overall accuracy (95% Wilson CI [87.1, 91.6]%). Correctly classified reconstructions achieve a mean confidence of 96.8% (95% bootstrap CI [96.0, 97.5]%), confirming that DeepExtractor effectively preserves the morphological characteristics recognized by Gravity Spy. Misclassified samples show lower confidence of 84.8%, suggesting that classifier confidence may serve as a useful quality veto.

TABLE IV: Gravity Spy classification results for DeepExtractor reconstructions ($n = 100$ samples per class, 700 total). Accuracy intervals use the Wilson score method; confidence intervals use a percentile bootstrap (10,000 resamples).

Class	Accuracy	95% Wilson CI	Mean conf.	95% bootstrap CI
Blip	100.0%	[96.3, 100.0]	0.993	[0.988, 0.997]
Fast Scattering	77.0%	[67.8, 84.2]	0.898	[0.863, 0.930]
Koi Fish	79.0%	[70.0, 85.8]	0.916	[0.885, 0.944]
Low Frequency Burst	100.0%	[96.3, 100.0]	0.998	[0.997, 1.000]
Scattered Light	83.0%	[74.5, 89.1]	0.970	[0.954, 0.982]
Tomte	96.0%	[90.2, 98.4]	0.982	[0.963, 0.996]
Whistle	92.0%	[85.0, 95.9]	0.993	[0.982, 0.999]
Overall (700)	89.6%	[87.1, 91.6]	0.968	[0.960, 0.975]

Table IV shows that class-level accuracy varies considerably. All Blip and Low-Frequency Burst glitches

are correctly classified, while Fast Scattering and Koi Fish achieve the lowest accuracies at 77% and 79%, respectively. Figure 2 shows the full confusion matrix. Since Gravity Spy supports additional classes beyond our seven training classes, the matrix is asymmetric. Most misclassifications occur between morphologically similar classes: among the 21 misclassified Koi Fish glitches, 14 were labelled as Extremely Loud, consistent with their exceptionally high SNRs (220–1270) overwhelming finer morphological details. The 23 misclassified Fast Scattering samples are distributed across several related classes (Scattered Light, Tomte, Repeating Blips), reflecting the known morphological similarity between low-frequency scattering noise types. Overall, the misclassification patterns are morphologically explainable and do not indicate systematic failure of the reconstruction.

2. Unsupervised clustering with t-SNE and UMAP

Figure 3 shows three-dimensional t-SNE (left) and UMAP (right) embeddings, fitted on all 31,511 available DeepExtractor reconstructions, visualized from three complementary viewpoints. The reconstructions form well-separated clusters that align closely with the Gravity Spy labels, demonstrating that morphologically distinct glitch types occupy distinct regions of the time-domain latent space. Partial overlap between the Blip and Koi Fish clusters is visible, consistent with prior observations that these classes may share a physical origin [59]. The good clustering structure validates that the 31,511 DeepExtractor reconstructions used to train GlitchGAN are morphologically representative of seven distinct classes and are not dominated by reconstruction artifacts or label noise.

B. Validation of GlitchGAN synthetic glitches

1. Classification evaluation using Gravity Spy

To quantify synthetic glitch realism we follow a similar approach to the DeepExtractor evaluation, injecting 100 generated glitches per class into quiet Hanford O3 background noise and classifying them with Gravity Spy. The resulting confusion matrix shown in Figure 4 reflects classifier predictions and confidence scores in brackets, with the full per-class breakdown summarized in Table V. The overall classification accuracy is 79.3% (95% Wilson CI [76.1, 82.1]%). While this is lower than the results for DeepExtractor reconstructions, the misclassifications are consistent with known limitations of Gravity Spy and the use of a fixed injected SNR according to the mean SNR of each glitch class.

Most misclassifications occur in the Koi Fish and Scattered Light classes. While only 70% of the generated Koi Fish samples were correctly classified, the majority of the remaining samples were misidentified as Ex-

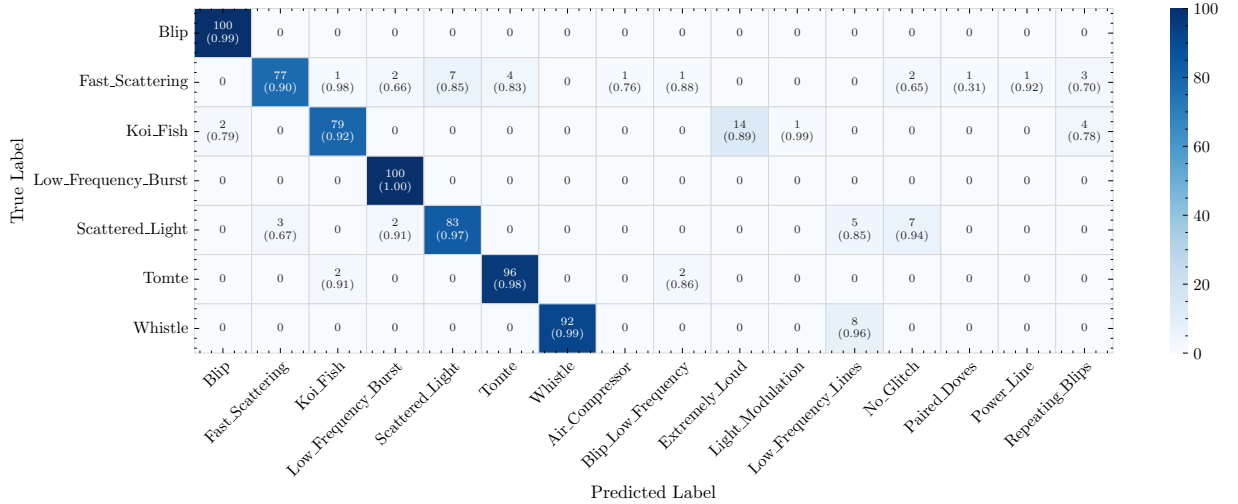


FIG. 2: Confusion matrix for Gravity Spy classification of 100 DeepExtractor reconstructions per class (700 total). Each cell shows the number of predictions with the mean Gravity Spy confidence in brackets.

tremely Loud, Light Modulation and Blip. This behaviour is consistent with known morphological similarities between Koi Fish and these glitch classes, which may arise from related physical mechanisms differing primarily in SNR [59]. Moreover, changing the injected SNR of the generated Koi Fish samples to 150 raises the classification accuracy to 79% with an average confidence of $\sim 95\%$, further illustrating the dependence of Gravity Spy classifications on the injected SNR.

This behaviour is further supported by injecting glitches at an SNR of 20. In this lower-SNR regime, none of the generated Koi Fish samples were correctly classified. Instead, they were misidentified as other morphologically similar classes. A comparable SNR dependence is observed for the Whistle class, where 72% of glitches were correctly identified. Injecting the generated Whistle glitches at an SNR of 50 yields an accuracy of 94%, whereas injecting them at an SNR of 20 leads to frequent misclassification as No Glitch.

For Scattered Light, Gravity Spy often predicts Fast Scattering, which is expected given their related scattering origins at low frequency. This was also observed for DeepExtractor’s original reconstructions of both Fast Scattering and Scattered Light shown in Figure 2.

It is also important to note that some misclassifications may arise from stochastic interactions between the injected glitches and the specific noise realization into which they are injected, which can differ from the original conditions of the glitch sample. Additionally, labelling uncertainties in the Gravity Spy training set may contribute, and the discarded phase information in Q -scans may play a role—an issue we discuss further below.

Finally, we report that GlitchGAN generates 1000 glitches in approximately 22 seconds on a CPU, demonstrating the model’s efficiency for large-scale glitch synthesis.

TABLE V: Gravity Spy classification results for GlitchGAN-generated glitches ($n = 100$ samples per class, 700 total). Accuracy intervals use the Wilson score method; confidence intervals use a percentile bootstrap (10,000 resamples).

Class	Accuracy	95% Wilson CI	Mean conf.	95% bootstrap CI
Blip	85.0%	[76.7, 90.7]	0.976	[0.955, 0.993]
Fast Scattering	73.0%	[63.6, 80.7]	0.945	[0.917, 0.970]
Koi Fish	70.0%	[60.4, 78.1]	0.939	[0.915, 0.961]
Low Frequency Burst	100.0%	[96.3, 100.0]	1.000	[0.999, 1.000]
Scattered Light	59.0%	[49.2, 68.1]	0.910	[0.879, 0.939]
Tomte	96.0%	[90.2, 98.4]	0.992	[0.981, 1.000]
Whistle	72.0%	[62.5, 79.9]	0.892	[0.851, 0.930]
Overall (700)	79.3%	[76.1, 82.1]	0.957	[0.948, 0.965]

2. Global morphological similarity via UMAP

To assess whether GlitchGAN captures the overall structure of the real glitch distribution, we project both real and synthetic glitch samples into a two-dimensional latent space using UMAP [43]. The embedding is fitted jointly to real and synthetic samples, using 2,000 real glitches per class (14,000 total) and 2,000 synthetic glitches per class (14,000 total), and the resulting embedding is visualized from three complementary perspectives in Figure 5, showing two UMAP dimensions in each plot.

Figure 5 shows 800 real and 800 generated samples per class (11,200 total), randomly selected from the 28,000 samples used to fit the UMAP embedding.

Corresponding individual class-wise UMAP projections using the same embedding are shown in Figure 6, and provide detail to compare real vs. synthetic samples for each class. Strong overlap between real (blue) and synthetic (green) samples indicates that GlitchGAN has learned the general morphological structure of the different glitch types. The experiment embedding was fitted using different combinations of seeds and hyperparameters to ensure the patterns are consistent and not an

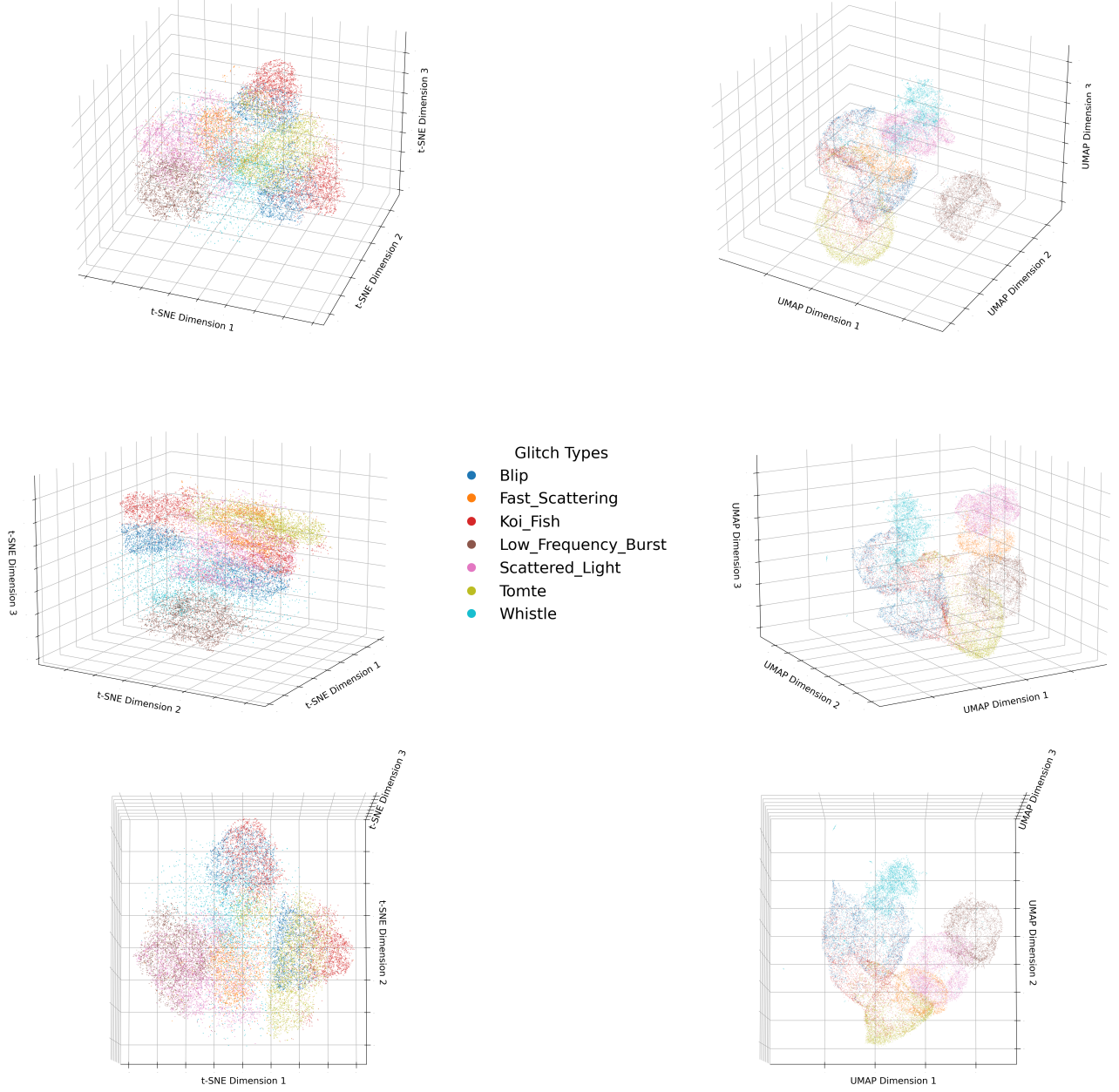


FIG. 3: Three-dimensional *t*-SNE (left) and UMAP (right) embeddings of DeepExtractor reconstructions across three viewpoints. Colours correspond to Gravity Spy glitch classes. Well-separated clusters demonstrate that reconstructions preserve class-specific morphological structure in the time domain. The partial overlap between Blip and Koi Fish (visible in the middle row) is consistent with their known morphological proximity.

artifact of one particular embedding. Similar patterns were observed across all iterations, confirming that the results are robust to the choice of hyperparameters and random seed. The different configurations are described in Appendix C and the results of the different iterations can be found in our online repository.

For most classes (Blip, Fast Scattering, Scattered Light, Tomte), synthetic and real samples show close

clustering with substantial overlap. The Whistle class exhibits the most noticeable divergence, with synthetic samples forming a partially separate cluster, suggesting that GlitchGAN captures the dominant Whistle morphology but does not fully explore the full morphological diversity within this class. This may reflect limitations of the fixed GAN architecture when applied to this particular class without hyperparameter tuning, which is

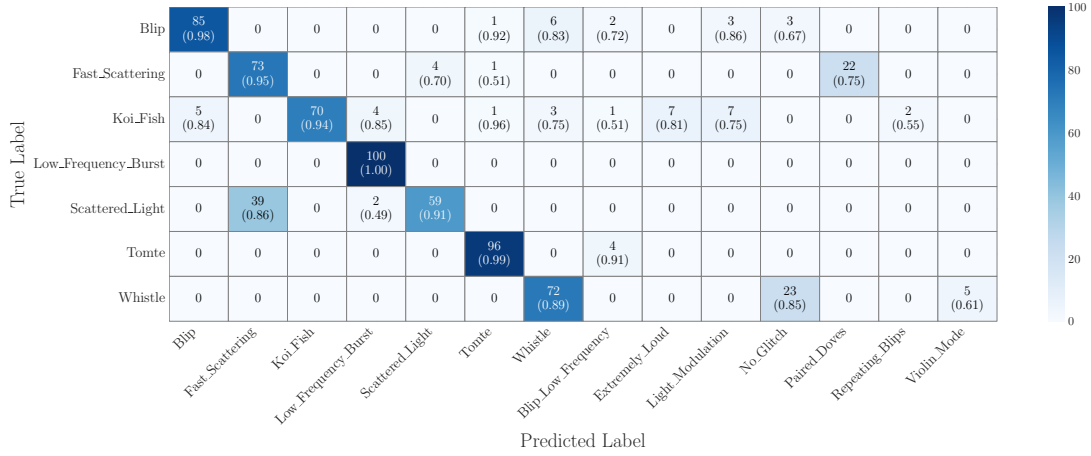


FIG. 4: Confusion matrix showing Gravity Spy classifications of glitches generated by GlitchGAN, with 100 samples per class. Each cell indicates the number of samples predicted for each class, with the average Gravity Spy confidence shown in brackets.

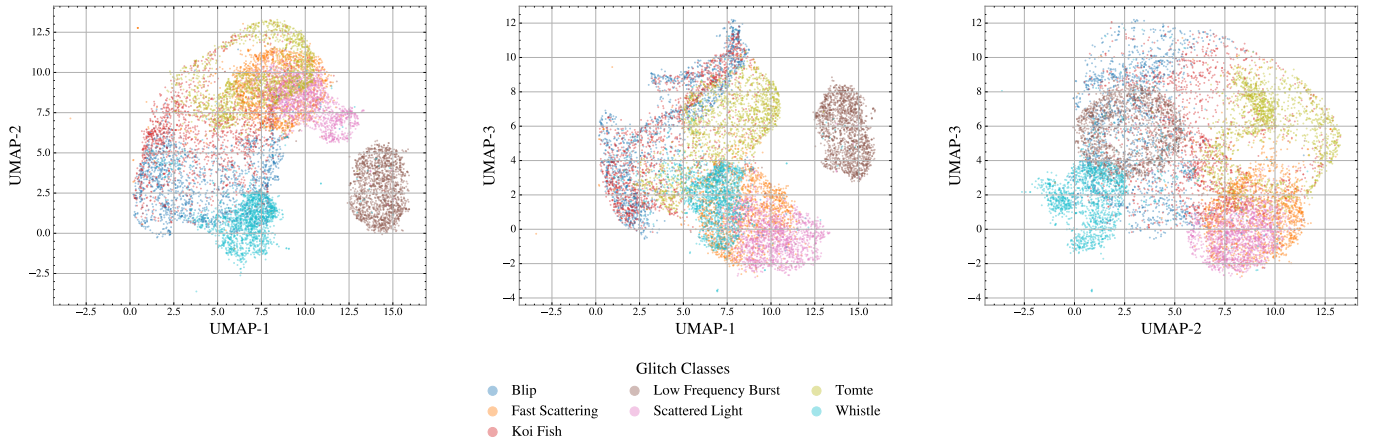


FIG. 5: Three complementary views of two-dimensional UMAP embeddings showing both real and generated glitch samples. Each color corresponds to one of the seven glitch classes used during training, with 2,000 real and 2,000 generated samples for each class. The corresponding real vs synthetic class-wise UMAP projections using the same embedding are shown in Figure 6.

discussed in more detail below.

3. Downstream Classification of real vs. synthetic glitches

The CNN classifier distinguishes real from generated glitches with high accuracy of 95.7% (95% CI [94.4, 96.7%]). A linear logistic regression baseline model fit directly on the raw waveform, with its L2 regularization strength selected via the validation split, is less capable of this discrimination, achieving 73.6% accuracy, substantially lower than the corresponding CNN accuracy, although still well above random chance. This mirrors a broader finding in the generative modelling literature: purpose-built classifiers routinely detect GAN-generated content with high accuracy even when it is otherwise realistic by coarser measures [60]. Given the results of the previous experiments, we interpret this result not as evidence that GlitchGAN’s synthetic glitches are morpho-

logically dissimilar from real ones, but as a caveat on the resolution at which consistency holds against Deep-Extractor’s reconstructions. For example, small artifacts in the generated glitches may be sufficient for a CNN to distinguish real from synthetic glitches even when they are otherwise morphologically similar. GlitchGAN reproduces class-level morphology and gross structural similarity faithfully, but synthetic glitches remain distinguishable from real ones under sufficiently targeted, fine-grained scrutiny.

Repeating this test with real and generated glitches injected into realistic detector noise reduces discriminability substantially. At the epoch-210 checkpoint, the CNN’s accuracy drops to 66.0% (95% CI [63.3, 68.5%]), while the linear baseline’s accuracy falls to 58.9% (95% CI [56.2, 61.6%]). This is consistent with the interpretation above: the CNN’s advantage relied substantially on a fairly specific, consistent signature that is far less reliable once real and generated glitches are injected

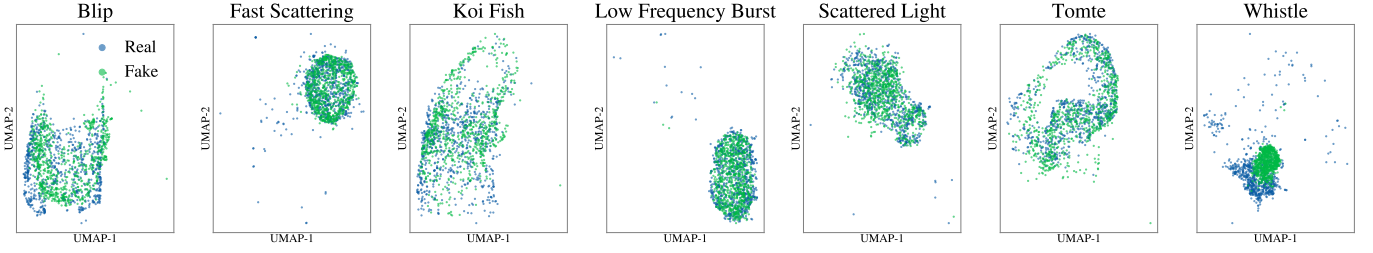


FIG. 6: Two-dimensional UMAP embeddings for each glitch class, comparing real (blue) and GlitchGAN-generated (green) samples for the same UMAP embedding shown in Figure 5. Each panel corresponds to one glitch type, showing the same 800 real and 800 generated samples per class plotted in Figure 5. The close overlap between real and synthetic distributions across most classes demonstrates that GlitchGAN effectively learns class-specific morphological features, though some variation remains for more complex glitch morphologies.

into realistic noise at per-sample SNRs drawn from the true empirical distribution (Section III D 3), rather than presented directly in the clean, noise-free representation used above. Under conditions that more closely resemble actual detector data, physical consistency between real and synthetic glitches therefore holds more strongly than the clean-domain result alone would suggest.

4. Multi-class classification and data augmentation

Table VI shows seven-class classification accuracy, with real and generated glitches injected into realistic detector noise, across three real-data scarcity levels: abundant (2,206 real samples total, the full held-out training pool at its natural 70/10/20 split, ~ 194 –349 per class), moderately scarce (316 real samples total, 28–50 per class), and severely scarce (95 real samples total, 8–15 per class): Within each class two augmentation-target scales are shown: a modest target matching each pool’s own majority-class count, and an aggressive target of 5,000 samples per class.

At the data-abundant end of this scarcity sweep – the full held-out real training pool at its natural 70/10/20 split (2,206 real samples total, ~ 194 –349 per class) – augmentation provides at most a modest benefit. At the majority-class target (~ 349 /class), none of bootstrap-duplication (89.8%), augmentation (91.4%), or synthetic-only training (89.0%) differ significantly from the unaugmented baseline (90.6%). Scaled up to 5,000 samples per class, bootstrap-duplication (92.2%), augmentation (92.4%), and synthetic-only training (90.9%) are all nominally higher than baseline, though none reach statistical significance at this test-set size.

To assess whether this changes as real data becomes scarcer, we repeat the experiment with a moderately scarce real training pool (316 samples total, 28–50 per class; middle third of Table VI). At the majority-class target (~ 50 /class), none of bootstrap-duplication (89.6%), augmentation (90.5%), or synthetic-only training (89.3%) differ significantly from the unaugmented baseline (90.5%), consistent with the classification task already being close to its accuracy ceiling with this

much real data available. Scaled up to 5,000 samples per class, a different pattern emerges: bootstrap-duplication becomes significantly *worse* than the unaugmented baseline (86.9% vs. 90.5%), while GlitchGAN-augmented and synthetic-only training remain statistically indistinguishable from it (90.6% and 90.7%, respectively). This harmful-duplication effect was absent at the data-abundant scale above, where the duplication factor was much milder ($\sim 18\times$ versus roughly 100 – $180\times$ here), suggesting the harm from naive duplication is tied to how extreme the duplication factor is, rather than being a general property of oversampling real data to a large nominal training set size. Even so, synthetic augmentation itself remains a *safe* choice in this regime, providing no measurable benefit but no harm either, unlike naive duplication.

Finally, at the most severely scarce end of this sweep, the picture depends strongly on target scale. At the modest majority-class target (~ 15 /class), bootstrap-duplication (85.1%) and augmentation (86.5%) are statistically indistinguishable from the unaugmented baseline (85.7%), while training on synthetic data alone is significantly *worse* (80.2%, non-overlapping 95% CI) – unsurprising, since this configuration discards the handful of real anchors entirely in favour of an equally small, purely synthetic set. Scaling the same three strategies up to 5,000 samples per class inverts this pattern: augmentation (91.0%) and synthetic-only training (91.8%) both become significantly better than baseline (non-overlapping 95% CIs), while bootstrap-duplication (87.4%) still shows no significant improvement. Since the augmented and duplicated configurations at this scale are matched in total training set size, this isolates the source of the improvement: it is GlitchGAN’s genuine morphological diversity, not merely a larger nominal training set, that drives the gain – and synthetic augmentation only pays off once deployed at sufficient scale.

Together, these results demonstrate a concrete, practical use case for GlitchGAN: in scenarios where real labelled examples of a glitch class are severely scarce – for instance, a newly-identified or infrequently-occurring noise transient – synthetic augmentation substantially improves downstream classifier performance, provided it

Scarcity level	Training set	Accuracy	95% CI
Abundant (70/10/20 split, $n = 2206$, ~ 194 – 349 /class)	Unaugmented	90.6%	[88.1, 92.7]%
	Bootstrap-duplicated @ majority (~ 349 /class)	89.8%	[87.2, 92.0]%
	Augmented @ majority	91.4%	[89.0, 93.4]%
	Synthetic-only @ majority	89.0%	[86.3, 91.2]%
	Bootstrap-duplicated @ 5000/class	92.2%	[89.8, 94.1]%
	Augmented @ 5000/class	92.4%	[90.0, 94.2]%
	Synthetic-only @ 5000/class	90.9%	[88.4, 92.9]%
Moderately scarce (10/20/70 split, $n = 316$, 28 – 50 /class)	Unaugmented	90.5%	[89.2, 91.7]%
	Bootstrap-duplicated @ majority (~ 50 /class)	89.6%	[88.2, 90.8]%
	Augmented @ majority	90.5%	[89.2, 91.7]%
	Synthetic-only @ majority	89.3%	[87.9, 90.5]%
	Bootstrap-duplicated @ 5000/class	86.9% [†]	[85.5, 88.3]%
	Augmented @ 5000/class	90.6%	[89.3, 91.8]%
	Synthetic-only @ 5000/class	90.7%	[89.4, 91.8]%
Severely scarce (3/20/77 split, $n = 95$, 8 – 15 /class)	Unaugmented	85.7%	[84.2, 87.0]%
	Bootstrap-duplicated @ majority (~ 15 /class)	85.1%	[83.6, 86.5]%
	Augmented @ majority	86.5%	[85.1, 87.8]%
	Synthetic-only @ majority	80.2% [†]	[78.6, 81.8]%
	Bootstrap-duplicated @ 5000/class	87.4%	[86.0, 88.6]%
	Augmented @ 5000/class	91.0%	[89.8, 92.0]%
	Synthetic-only @ 5000/class	91.8%	[90.6, 92.8]%

TABLE VI: Seven-class classification accuracy under realistic detector-noise injection, across three real-data scarcity levels and two augmentation-target scales: a modest target matching each training pool’s own majority-class count, and an aggressive target of 5,000 samples per class. **Bold** entries are significantly better, and entries marked [†] significantly worse, than that scarcity level’s unaugmented baseline (non-overlapping 95% Wilson CI).

is deployed at sufficient scale, and does so more reliably than naive resampling of the same limited real data, which offers no benefit at matched scale and becomes actively harmful at large nominal scale in this regime. As real data becomes more abundant, this benefit diminishes and eventually vanishes entirely, consistent with the classification task saturating quickly regardless of data source. Naive duplication’s harm at large nominal scale is itself scarcity-dependent: it appears only when the real pool is small enough that reaching a large training set requires an extreme duplication factor, and does not reappear once enough real data is already available that the same nominal scale is reached more moderately. Nevertheless, GlitchGAN augmentation is never harmful, showing that it effectively learned the morphological structure of the real glitches.

5. Generating hybrid glitches via class interpolation

A key advantage of class-conditional generative models is the ability to produce “hybrid” or transitional morphologies by sampling mixed class vectors.

We explore two sampling strategies:

1. **Simplex mixing:** The class vector is normalized such that all coefficients sum to 1, i.e., sampled from a $k = 7$ simplex. This constrains mixing proportions to sum to 100%.

2. **Uniform mixing:** Each class coefficient is independently sampled from $[0, 1]$ without normalization constraint, allowing more extreme interpolations.

Figure 7 shows glitch examples from both mixing strategies, demonstrating intermediate morphologies. As shown, samples from these datasets exhibit mixed morphological characteristics drawn from multiple glitch classes as per the sampled class vector. The UMAP projection of mixed samples in Figure 8 confirms that these hybrid samples smoothly interpolate between pure-class point clouds, with uniform mixing exploring the broadest region of the latent space.

It is important to note that these hybrid glitches do not represent real transitional glitch mechanisms. Rather, they are synthetic class mixtures produced by interpolating the generative model’s class-conditioning input, which the model maps to smoothly and continuously varying morphologies. While hybrid samples are not further analyzed in this study, they have potential applications for pipeline validation and data augmentation for glitch detection or classification systems.

C. Limitations of magnitude-only representations

We further evaluated the DiT model as a comparison; the full Gravity Spy and UMAP results are shown in Appendix F. DiT-generated samples overlap with real

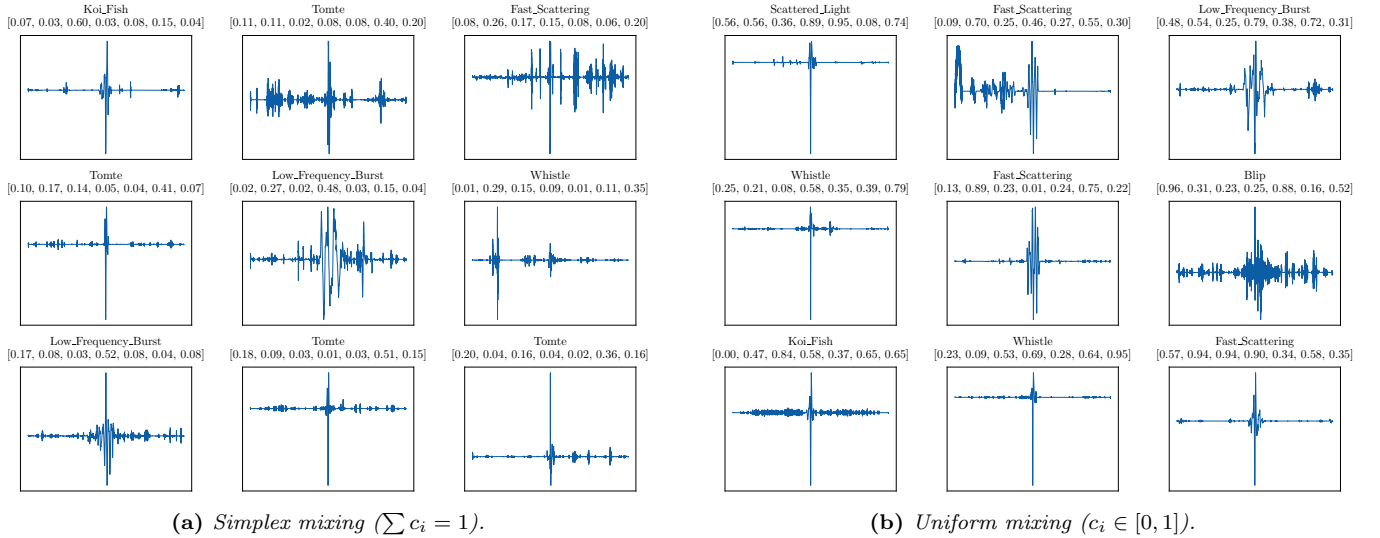


FIG. 7: Examples of hybrid glitch waveforms generated by GlitchGAN using two class-conditioning strategies. (a) Simplex mixing samples each class vector from a $k = 7$ simplex, ensuring coefficients sum to one. (b) Uniform mixing samples each coefficient independently from a uniform distribution in $U[0, 1]$. The corresponding sampled class vectors are shown above each plot, with the dominant glitch label also shown. Both methods produce hybrid morphologies interpolating between glitch classes, demonstrating GlitchGAN’s ability to explore smooth transitions across the learned latent class space.

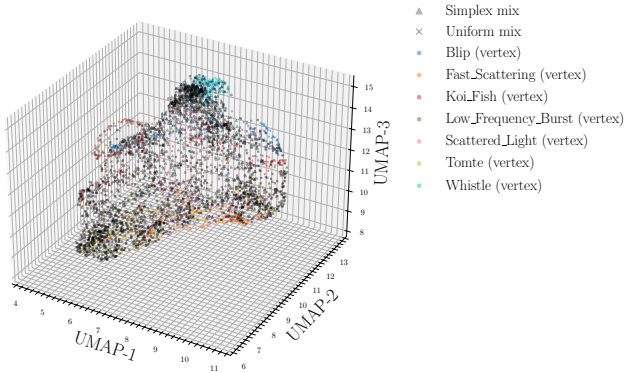


FIG. 8: Three-dimensional UMAP embedding showing the distribution of class-mixed glitch samples relative to the vertex (pure-class) glitches. Vertex samples are coloured by their respective glitch class, while class-mixed samples generated using simplex and uniform class-conditioning vectors are shown in black. The smooth interpolation of mixed samples between class clusters illustrates how GlitchGAN captures continuous transitions across glitch morphologies in the latent class space.

glitches primarily for low-frequency classes (e.g., Fast Scattering, Scattered Light), but show reduced diversity and separable clusters for most other classes where GlitchGAN was more successful.

A notable example is the Low Frequency Burst class: UMAP reveals two clearly distinct clusters (real vs. synthetic), yet Gravity Spy classifies 87% of DiT-generated samples correctly with 97% mean confidence. Figure 9 illustrates why: while the time-domain waveforms of real

and DiT-generated Low Frequency Bursts are clearly distinct, their magnitude Q -scan representations appear similar, explaining the classifier’s high confidence despite the physically unrealistic time-domain morphology.

This finding underscores a critical limitation: classifiers operating on magnitude-only representations cannot reliably assess glitch realism. Complementary validation methods preserving phase information (such as UMAP on time-domain waveforms) are essential for ensuring synthetic glitches are suitable for detector simulations.

Overall, while the DiT model demonstrates that latent diffusion is a viable approach to conditional time-domain glitch generation, its ability to reproduce full morphological diversity—especially for mid- and high-frequency content—is limited relative to GlitchGAN. It is important to note that the DiT model implemented serves only to highlight the limitation in magnitude-only classification. Better diffusion architectures and training strategies may improve performance beyond GlitchGAN and other GAN architectures.

V. DISCUSSION

A. Performance assessment and limitations

GlitchGAN successfully demonstrates time-domain synthesis of multiple glitch classes reconstructed by DeepExtractor. Substantial UMAP overlap between real and synthetic samples, combined with accurate Gravity Spy classification of generated glitches indicates morphological fidelity of generated glitches. However, there

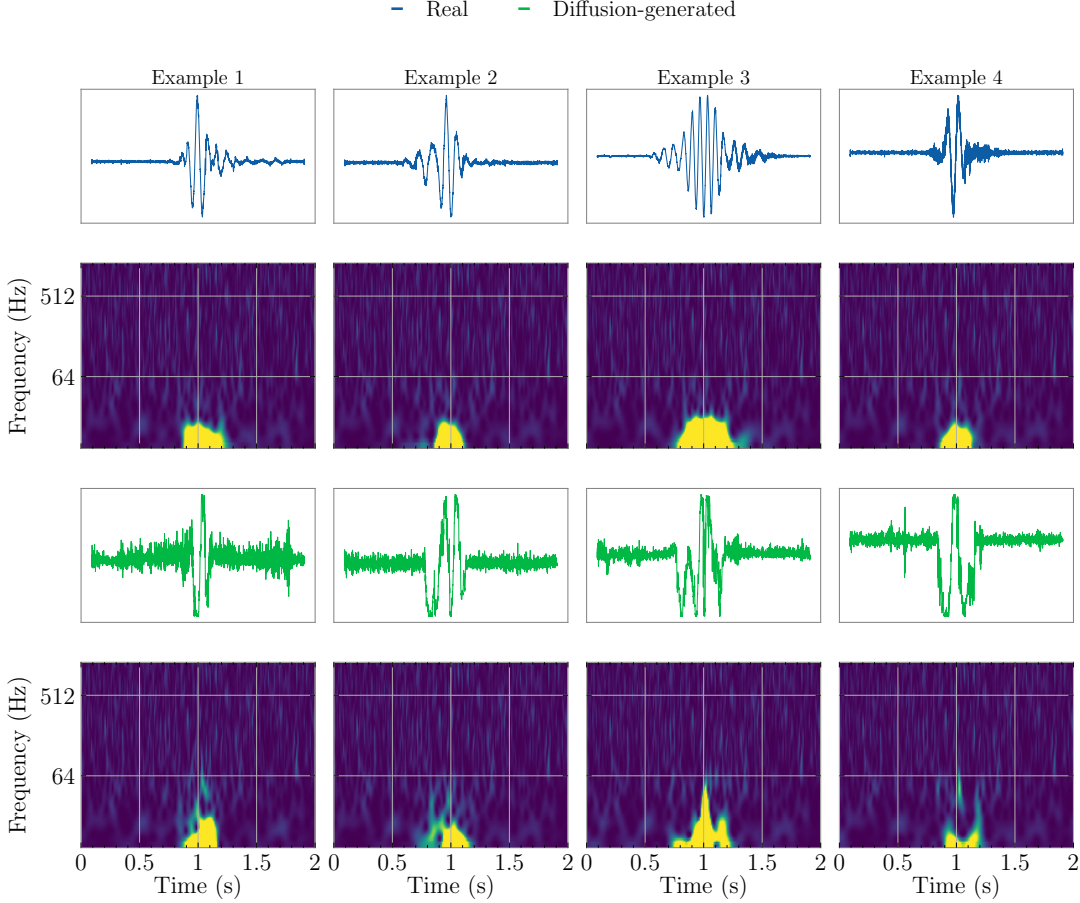


FIG. 9: Four examples each of real (top two rows) and DiT-generated (bottom two rows) Low Frequency Burst glitches, showing both the time-domain waveform and the magnitude Q -scan representation. The time-domain waveforms are clearly distinct, yet the Q -scans appear similar, explaining why Gravity Spy classifies the DiT samples correctly with high confidence.

exists several limitations and caveats which we address here.

Relatedly, Gravity Spy classification and UMAP embeddings are comparatively coarse validation metrics, operating at the level of class-wide structure rather than fine-grained statistical detail. A dedicated real-vs-generated CNN classifier (Section III D 3) detects a residual statistical signature distinguishing synthetic from real glitches with high accuracy in the clean, noise-free domain, indicating that subtle, nonlinear differences remain even where these coarser methods show close agreement. This detectability is substantially, though not completely, reduced once glitches are considered under realistic detector-noise conditions, suggesting the residual signature is most apparent in idealized representations rather than in the conditions under which synthetic glitches would actually be deployed.

Partial failure was observed for the Whistle class, possibly due to insufficient regularization or architectural constraints on receptive field size. For example, smaller kernel sizes in generator and discriminator layers may be needed to capture the high-frequency content of Whistle

glitches (please see Appendix A for details).

Since GlitchGAN is trained on normalized waveforms, it does not learn an explicit SNR distribution, and thus generated glitches must be rescaled to a specified SNR for use in downstream applications. As shown in Section IV B 1, Gravity Spy classifications are strongly sensitive to the injected SNR — consistent with the fact that individual glitch classes are partly characterized by their SNR distribution. Appropriate rescaling is therefore an important consideration for simulations and pipeline validation studies using these synthetic samples. For example, one may choose to rescale generated glitches to match the SNR distribution of the real glitches within a given class, or to a fixed SNR for specific applications such as testing pipeline sensitivity at particular SNR thresholds. To ensure fidelity for downstream applications, Gravity Spy can be used to verify that the rescaled glitches are classified as the intended class with high confidence.

Given the empirical relationships that exist between SNR and glitch morphology, at least for certain classes (e.g. Blip vs. Koi Fish [59]), it is important to con-

sider the implications of rescaling generated glitches to a fixed SNR. Normalizing glitches to a fixed SNR, or to the class-wide SNR distribution, may alter the morphology of the generated glitches relative to the real population, which is an important consideration for downstream applications. A more realistic rescaling strategy could instead use a k -nearest-neighbours (kNN) approach, in which each generated glitch is assigned an SNR sampled from the k real training glitches nearest to it in morphology space, thereby preserving the class-internal SNR-morphology correlations that a single fixed or class-averaged SNR would erase. Jointly modelling glitch morphology and SNR is a more complete, though more complicated, direction for future work to achieve fully realistic synthetic glitches, which could be achieved by extending the class-conditional generator to include an additional SNR-conditioning input.

While DeepExtractor shows great potential for glitch reconstruction, it is not perfect and may introduce artifacts or distortions in the reconstructed glitches. These artifacts may propagate into the GlitchGAN training data, potentially affecting the quality of generated glitches, which is difficult to quantify for real glitches since we lack ground truth glitch waveforms. However, the strong agreement between real and synthetic glitches in UMAP embeddings and Gravity Spy classifications suggests that any artifacts introduced by DeepExtractor are not dominant in the training data. That being said, should a more accurate glitch reconstruction method become available in the future, it could be used to retrain GlitchGAN and potentially improve the quality of generated glitches.

Finally, we note that no hyperparameter optimization was performed for this specific dataset; the model is evaluated “out-of-the-box” from previous work (except for the dimensionality of the class vector), representing a conservative assessment of GlitchGAN’s potential. Tuning the parameters of the generator and discriminator architectures, as well as the training procedure, could further improve performance, particularly for the more challenging Whistle class.

B. Practical applications

The synthetic glitch population produced by GlitchGAN is immediately useful for:

1. **Mock data challenges (MDCs):** Enriching realistic detector simulations with synthetic instrumental noise
2. **Detector characterization:** Validating glitch detection, mitigation, and subtraction algorithms
3. **Training data augmentation:** Supplementing limited real glitch samples with GlitchGAN-generated samples, demonstrated in Section IV B 4

to substantially improve downstream classification accuracy in data-scarce regimes

4. **Pipeline robustness testing:** Assessing pipeline sensitivity and false-alarm rates in the presence of instrumental noise morphologies

C. Recommendations for future work

There are several promising directions for future research building on this work. Firstly, hyperparameter optimization and architectural refinements could be performed to further improve the quality of generated data, particularly for the more challenging Whistle class. Architecture improvements may include the incorporation of attention mechanisms, smaller convolutional kernel sizes or dilated convolutions for multi-scale feature extraction.

Ultimately, the goal is to develop a comprehensive generative model capable of synthesizing the entire range of glitch morphologies in GW detectors with high fidelity, enabling robust simulations and pipeline validation for current and future GW detectors. Future work will iteratively include more glitch classes, starting with more common and progressively incorporating less common ones.

It would be beneficial to develop more sophisticated validation metrics that explicitly evaluate phase consistency, not just magnitude-level morphology, to ensure synthetic glitches are suitable for detector simulations. Such methods will be beneficial for evaluating the realism of synthetic glitches and guiding future improvements in generative modeling approaches.

While class conditional generation is a powerful tool, it may also be desirable to develop conditional generation strategies allowing control over glitch duration, SNR, and frequency content independently from class morphology. This would enable more flexible synthesis of glitches for specific applications, such as testing pipeline sensitivity to particular frequency bands or durations.

Finally, integrating these features with mock data challenge pipelines for rapid deployment in detector simulations will be an important step towards practical utility of these synthetic glitch populations for the GW data analysis community.

VI. CONCLUSIONS

We have demonstrated that a class-conditional deep generative model called GlitchGAN can effectively synthesize realistic GW detector glitches directly in the time domain at speed, generating 1000 glitch samples in under 22 seconds on a CPU. By leveraging high-fidelity reconstructions from a glitch reconstruction framework called DeepExtractor and training a single unified GlitchGAN model based on the cDVGAN architecture across

seven glitch classes, we achieve synthetic samples with strong morphological agreement with real data. GlitchGAN’s synthetic glitch population offers practical utility for mock data challenges, detector characterization studies, and validation of detection pipelines.

We evaluated both DeepExtractor’s reconstructions used to train GlitchGAN and GlitchGAN’s synthetic data using two complementary approaches: a supervised classification analysis using Gravity Spy, a state-of-the-art glitch classification algorithm trained on an extensive labeled dataset, and an unsupervised, data-driven UMAP analysis to visualize and compare the class distributions of real and synthetic glitches in a projected three-dimensional space.

The Gravity Spy classification results for DeepExtractor reconstructions showed that 89.57% of the reconstructed glitches were correctly classified, with a mean classifier confidence of 96.77% for correct predictions. Misclassified samples showed a lower confidence of 84.84%, suggesting that classifier confidence can serve as a quality veto. Overall, the misclassification patterns are generally explainable, typically misclassifying as morphologically similar classes, and do not indicate systematic failure of the reconstruction. The unsupervised UMAP analysis of DeepExtractor reconstructions revealed well-separated clusters according to the Gravity Spy labels, demonstrating that the reconstructions preserve class-specific morphological structure in the time domain.

For GlitchGAN’s generated glitches, Gravity Spy classified most samples correctly with an overall accuracy of 79%. Misclassifications primarily occurred for the Koi Fish and Scattered Light classes, which can largely be attributed to morphological similarities between similar classes and Gravity Spy’s classification being dependent on SNR. For instance, Scattered Light glitches were often misclassified as Fast Scattering, while Koi Fish are often misclassified as Extremely Loud or Light Modulation. These overlaps are expected, as the classes share similar time-frequency signatures. This is confirmed by changing the injected SNR to 150 for Koi Fish samples and 50 for the Whistle class, where 79% and 94% are correctly classified, respectively.

The UMAP results showed that GlitchGAN-generated glitches generally occupy the same regions as real data in the embedded UMAP space, indicating that the model captures much of the morphological diversity within each glitch class. One exception is the Whistle class, which consists of the highest-frequency features in the dataset. For this class, synthetic samples formed a partially distinct cluster from the real data, suggesting that GlitchGAN did not fully capture the fine-scale, high-frequency structure characteristic of these glitches. Future work could address this by incorporating smaller convolutional kernels in the network (balanced against the need for long-range temporal modelling), dilated convolutions to better capture hierarchical structure, or attention mechanisms to enhance global feature representation.

Beyond these coarser validation methods, we further

tested physical consistency using a dedicated classifier trained specifically to discriminate real glitches from GlitchGAN-generated ones. This classifier reliably detects synthetic samples in a clean, noise-free representation, indicating that subtle statistical differences remain even where class-level and structural similarity are strong. However, this detectability narrows substantially once glitches are injected into realistic detector noise, suggesting that physical consistency between real and synthetic glitches holds more strongly under conditions resembling actual detector data than the clean-domain result alone would suggest.

Despite this residual detectability, we demonstrated a concrete practical use case for GlitchGAN: augmenting a severely data-scarce real training set with synthetic samples substantially improved downstream multi-class glitch classification accuracy, far exceeding what naive duplication of the same limited real data could achieve. This indicates that GlitchGAN’s synthetic glitches carry genuine, useful morphological information beyond simply increasing dataset size, and that this benefit is most pronounced precisely where it matters most: scenarios with only a handful of labelled examples of a given glitch class, such as newly-identified or infrequently-occurring noise transients.

While it is not investigated explicitly how reconstruction artifacts from DeepExtractor may affect GlitchGAN’s performance, the strong agreement between real and synthetic glitches in UMAP embeddings and Gravity Spy classifications suggests that any artifacts introduced by DeepExtractor are not dominant in the training data, and GlitchGAN is effective at learning the distributions reconstructed by DeepExtractor. Furthermore, should a more accurate glitch reconstruction method become available in the future, it could be used to retrain GlitchGAN and potentially improve the quality of generated glitches.

Finally, this study highlights a fundamental limitation in using purely magnitude-based Q -scan representations for supervised glitch classification. We observed that a less robust diffusion-based model, which produces unrealistic Low Frequency Burst glitches in the time domain, can still yield correct Gravity Spy labels with high confidence. Although the time-domain waveforms of real and synthetic samples are clearly distinct, their Q -scan representations appear similar, explaining the classifier’s high confidence. This discrepancy is evident in the corresponding UMAP analysis of time-domain data, which reveals two distinct clusters separating real and synthetic samples. This finding highlights the importance of preserving phase information and integrating unsupervised analyses to complement glitch characterization efforts.

ACKNOWLEDGEMENTS

This research has made use of data or software obtained from the Gravitational Wave Open Science Cen-

ter, a service of the LIGO Scientific Collaboration, the Virgo Collaboration, and KAGRA. This material is based upon work supported by NSF’s LIGO Laboratory which is a major facility fully funded by the National Science Foundation, as well as the Science and Technology Facilities Council (STFC) of the United Kingdom, the Max-Planck-Society (MPS), and the State of Niedersachsen/Germany for support of the construction of Advanced LIGO and construction and operation of the GEO600 detector. Additional support for Advanced LIGO was provided by the Australian Research Council. Virgo is funded, through the European Gravitational Observatory (EGO), by the French Centre National de Recherche Scientifique (CNRS), the Italian Istituto Nazionale di Fisica Nucleare (INFN) and the Dutch Nikhef, with contributions by institutions from Belgium, Germany, Greece, Hungary, Ireland, Japan, Monaco, Poland, Portugal, Spain. KAGRA is supported by Ministry of Education, Culture, Sports, Science and Technology (MEXT), Japan Society for the Promotion of Science (JSPS) in Japan; National Research Foundation (NRF) and Ministry of Science and ICT (MSIT) in Korea; Academia Sinica (AS) and National Science and Technology Council (NSTC) in Taiwan. The authors are grateful for computational resources provided by the LIGO Laboratory and supported by the National Science Foundation Grants No. PHY-0757058 and No. PHY-0823459.

DATA AVAILABILITY

The training dataset of DeepExtractor reconstructions used in this work, along with pretrained GlitchGAN generator weights, is hosted on Hugging Face and can be downloaded directly via the `glitchgan` Python package (<https://github.com/tomdooney95/glitchgan>).

Appendix A: cDVGAN Architecture

Table VII shows the cDVGAN architecture featured in GlitchGAN.

Discriminator (3.7M parameters)					
Operation	Output shape	Kernel size	Stride	Dropout	Activation
Input	(8192)	–	–	0	–
Reshape	(64,128)	–	–	0	–
Convolutional	(64,128)	14	1	0.5	Leaky ReLU
Convolutional	(32,128)	14	2	0.5	Leaky ReLU
Convolutional	(16,256)	14	2	0.5	Leaky ReLU
Convolutional	(8,256)	14	2	0.5	Leaky ReLU
Convolutional	(4,512)	14	2	0.5	Leaky ReLU
Global Avg. Pooling	(512)	–	–	0	–
Dense	(128)	–	–	0	Linear
Dense	(1)	–	–	0	Linear
Class input	(7)	–	–	–	–
Class embedding	(128)	–	–	0	–
Scalar product	(1)	–	–	–	–
Dense + Scalar product	(1)	–	–	–	–
DV Discriminator (2.7M parameters)					
Operation	Output shape	Kernel size	Stride	Dropout	Activation
Input	(8191)	–	–	0	–
Dense	(256)	–	–	0	Leaky ReLU
Reshape	(8,32)	–	–	0	–
Convolutional	(8,64)	5	1	0.5	Leaky ReLU
Convolutional	(4,128)	5	2	0.5	Leaky ReLU
Convolutional	(2,256)	5	2	0.5	Leaky ReLU
Convolutional	(1,256)	5	2	0.5	Leaky ReLU
Global Avg. Pooling	(256)	–	–	0	–
Dense	(128)	–	–	0	Linear
Dense	(1)	–	–	0	Linear
Class input	(7)	–	–	–	–
Class embedding	(128)	–	–	0	–
Scalar product	(1)	–	–	–	–
Dense + Scalar product	(1)	–	–	–	–
Generator (3.8M parameters)					
Operation	Output shape	Kernel size	Stride	BN	Activation
Latent input	(100)	–	–	×	–
Class input	(7)	–	–	×	–
Class Dense	(32)	–	–	×	Linear
Concatenate	(132)	–	–	×	–
Dense	(4096)	–	–	×	ReLU
Reshape	(256,16)	–	–	×	–
Upsample (×2) + Conv.	(512,512)	18	1	✓	ReLU
Upsample (×2) + Conv.	(1024,256)	18	1	✓	ReLU
Upsample (×2) + Conv.	(2048,128)	18	1	✓	ReLU
Upsample (×2) + Conv.	(4096,64)	18	1	✓	ReLU
Upsample (×2) + Conv.	(8192,1)	18	1	✓	Linear
Flatten	(8192)	–	–	×	–
Optimizer	RMSprop ($\alpha = 0.0001$)				
Batch size	64				
Epochs	210				
Loss	Wasserstein				

TABLE VII: Architecture and hyperparameters of cDVGAN featured in GlitchGAN, for the 8192-sample glitch waveforms used here (10.2M parameters total). The model includes a base discriminator, derivative (DV) discriminator, and generator.

Appendix B: cDVGAN Loss

Figure 10 shows the training loss for each individual model component.

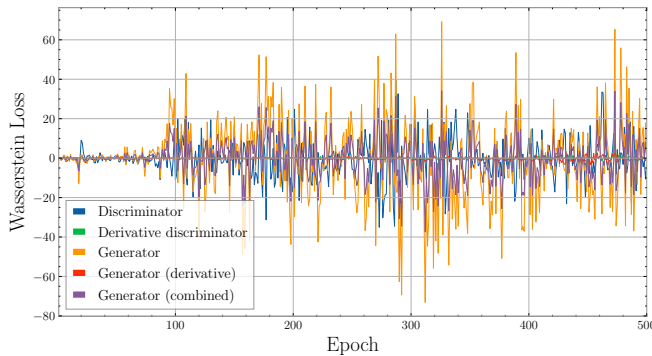


FIG. 10: GlitchGAN training loss over 500 epochs.

Appendix C: Hyperparameters for t-SNE and UMAP

Table VIII lists the hyperparameters used for the t-SNE and UMAP analyses.

TABLE VIII: Hyperparameters for t-SNE and UMAP. For UMAP, **bold** values give the primary configuration used for the Figures 5 and 6; the additional plain values were swept over to confirm the real/fake patterns are consistent and not an artifact of one particular embedding (9 total (*Random_state*, (*N_neighbors*, *Min_dist*)) combinations: seeds $\{0, 1, 2\} \times$ the three neighbor/distance pairs listed).

Algorithm	Hyperparameter	Value
t-SNE	N_components	3
	Perplexity	40
	Metric	euclidean
UMAP	(N_neighbors, Min_dist)	(15 , 0.6), (30, 0.3), (50, 0.1)
	Random_state	42 , 0, 1, 2
	N_components	3
	Metric	correlation

Appendix D: Downstream Classifier Architectures

Tables IX and X show the architectures of the binary real-vs-generated classifier (Section IIID3) and the seven-class multi-class classifier used for the data-augmentation experiment (Section IIID4), respectively. Figure 11 shows the empirical per-class SNR distributions used to inject both real and generated glitches into realistic detector noise for both experiments.

Operation	Output shape	Kernel size	Stride	Dropout or BN	Activation
Input	(8192, 1)	–	–	–	–
Conv1D	(2048, 32)	16	4	–	ReLU
Conv1D	(512, 64)	16	4	–	ReLU
Conv1D	(128, 128)	16	4	–	ReLU
GlobalAveragePooling1D	(128,)	–	–	–	–
Dense	(64,)	–	–	–	ReLU
Dropout	(64,)	–	–	0.3	–
Dense	(1,)	–	–	–	Sigmoid

TABLE IX: Architecture of the 1D CNN classifier used to discriminate between real and GlitchGAN-generated glitches in the downstream classification test. The network takes an 8192-sample time series as input and outputs a single scalar in $[0, 1]$, trained with a binary cross-entropy loss and Adam optimizer to predict real (1) vs. generated (0). Total trainable parameters: 172,897.

Appendix E: Empirical SNR distributions for noise injection

Figure 11 shows the empirical per-class SNR distributions used to inject both real and generated glitches into realistic detector noise for the downstream classification experiments (Sections IIID3 and IIID4).

Operation	Output shape	Kernel size	Stride	Dropout or BN	Activation
Input	(8192, 1)	—	—	—	—
Conv1D	(2048, 32)	16	4	—	ReLU
Conv1D	(512, 64)	16	4	—	ReLU
Conv1D	(128, 128)	16	4	—	ReLU
GlobalAveragePooling1D	(128,)	—	—	—	—
Dense	(64,)	—	—	—	ReLU
Dropout	(64,)	—	—	0.3	—
Dense	(7,)	—	—	—	Softmax

TABLE X: Architecture of the seven-class CNN classifier used for the multi-class data-augmentation experiment (Section III D 4). Identical convolutional backbone to the binary real-vs-generated classifier (Table IX), differing only in the final layer (seven-way softmax in place of a single sigmoid) and loss (sparse categorical cross-entropy). Total trainable parameters: 173,287.

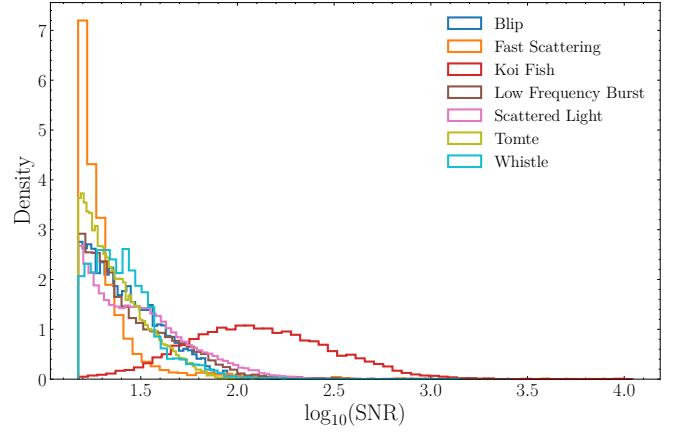


FIG. 11: Empirical per-class SNR distributions used for realistic noise injection, constructed from the GravitySpy O3a/O3b high-confidence catalogs under the same selection criteria as the training dataset (Section II). Distributions are heavily right-skewed; note the log-scale horizontal axis. Koi Fish is a visibly distinct, systematically louder population than the other six classes.

Appendix F: DiT evaluation results

Figures 12 and 13 show the full Gravity Spy and UMAP evaluation results for the DiT comparison model.

- [1] LIGO Scientific Collaboration and others, Advanced LIGO, *Class. Quant. Grav.* **32**, 074001 (2015), [arXiv:1411.4547 \[gr-qc\]](#).
- [2] F. Acernese *et al.* (VIRGO), Advanced Virgo: a second-generation interferometric gravitational wave detector, *Class. Quant. Grav.* **32**, 024001 (2015), [arXiv:1408.3978 \[gr-qc\]](#).
- [3] R. Abbott *et al.*, Gwtc-3: Compact binary coalescences observed by ligo and virgo during the second part of the third observing run, *Physical Review X* **13**, 10.1103/physrevx.13.041039 (2023).
- [4] The LIGO Scientific Collaboration *et al.*, Characterization of the ligo detectors during their third observing run, *Classical and Quantum Gravity* **38**, 135003 (2021).
- [5] D. Davis *et al.*, Ligo detector characterization in the second and third observing runs, *Classical and Quantum Gravity* **38**, 135014 (2021).
- [6] F. Acernese *et al.* (Virgo), Virgo detector characterization and data quality: results from the O3 run, *Class. Quant. Grav.* **40**, 185006 (2023), [arXiv:2210.15633 \[gr-qc\]](#).
- [7] T. Akutsu *et al.*, Overview of KAGRA: Calibration, detector characterization, physical environmental monitors, and the geophysics interferometer, *Progress of Theoretical and Experimental Physics* **2021**, 05A102 (2021), <https://academic.oup.com/ptep/article-pdf/2021/5/05A102/38109702/ptab018.pdf>.
- [8] D. Davis *et al.*, Rapid data quality investigations of gravitational-wave events with the data quality report builder toolkit (2026), [arXiv:2605.16183 \[astro-ph.IM\]](#).
- [9] B. P. Abbott *et al.* (LIGO Scientific, Virgo), Effects of data quality vetoes on a search for compact binary coalescences in Advanced LIGO's first observing run, *Class. Quant. Grav.* **35**, 065010 (2018), [arXiv:1710.02185 \[gr-qc\]](#).
- [10] L. Blackburn *et al.*, The LSC Glitch Group: Monitoring Noise Transients during the fifth LIGO Science Run, *Class. Quant. Grav.* **25**, 184004 (2008), [arXiv:0804.0800 \[gr-qc\]](#).
- [11] B. P. Abbott *et al.* (LIGO Scientific, Virgo), Characterization of transient noise in Advanced LIGO relevant to gravitational wave signal GW150914, *Class. Quant. Grav.* **33**, 134001 (2016), [arXiv:1602.03844 \[gr-qc\]](#).
- [12] C. Pankow *et al.*, Mitigation of the instrumental noise transient in gravitational-wave data surrounding gw170817, *Phys. Rev. D* **98**, 084016 (2018).
- [13] E. Payne, S. Hourihane, J. Golomb, R. Udall, D. Davis, and K. Chatziioannou, Curious case of gw200129: Interplay between spin-precession inference and data-quality issues, *Phys. Rev. D* **106**, 104017 (2022).
- [14] A. Gupta *et al.*, Possible causes of false general relativity violations in gravitational wave observations (2024), [arXiv:2405.02197 \[gr-qc\]](#).
- [15] B. P. Abbott *et al.* (LIGO Scientific Collaboration and Virgo Collaboration), Gw170817: Observation of gravitational waves from a binary neutron star inspiral, *Phys. Rev. Lett.* **119**, 161101 (2017).

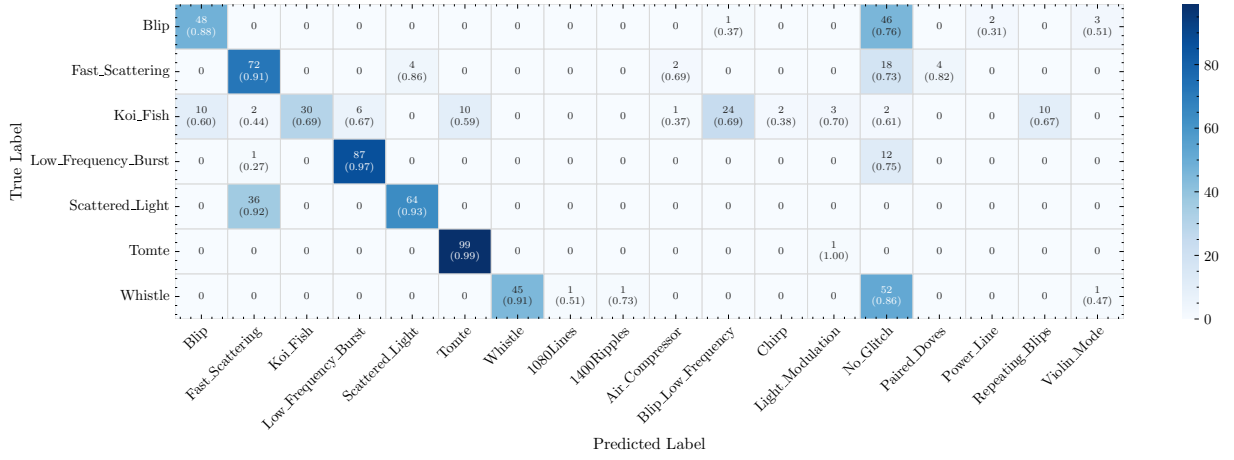


FIG. 12: Gravity Spy confusion matrix for samples generated by the DiT model (100 per class). Each cell shows the number of predictions and mean classifier confidence in brackets. Despite the poor UMAP overlap, Gravity Spy achieves high confidence classification for most classes, illustrating the limitation of magnitude-only validation.

- [16] B. P. Abbott *et al.*, Gw190425: Observation of a compact binary coalescence with total mass $\sim 3.4 m_{\odot}$, *The Astrophysical Journal Letters* **892**, L3 (2020).
- [17] R. Abbott *et al.*, Observation of gravitational waves from two neutron star–black hole coalescences, *The Astrophysical Journal Letters* **915**, L5 (2021).
- [18] S. Hild, M. Abernathy, F. Acernese, P. Amaro-Seoane, N. Andersson, K. Arun, F. Barone, B. Barr, M. Barsuglia, and M. B. e, Sensitivity studies for third-generation gravitational wave observatories, *Class. Quant. Grav.* **28**, 094013 (2011).
- [19] E. D. Hall, Cosmic explorer: A next-generation ground-based gravitational-wave observatory, *Galaxies* **10**, 10.3390/galaxies10040090 (2022).
- [20] F. Iacovelli *et al.*, Forecasting the detection capabilities of third-generation gravitational-wave detectors using GWFAST, *The Astrophysical Journal* **941**, 208 (2022).
- [21] V. Kalogera *et al.*, The next generation global gravitational wave observatory: The science book (2021).
- [22] J. Abadie *et al.*, Search for gravitational waves from compact binary coalescence in ligo and virgo data from s5 and vsr1, *Physical Review D* **82**, 10.1103/physrevd.82.102001 (2010).
- [23] J. Abadie *et al.*, All-sky search for gravitational-wave bursts in the first joint ligo-geo-virgo run, *Physical Review D* **81**, 10.1103/physrevd.81.102001 (2010).
- [24] A. E. Tolley *et al.*, ArchEnemy: removing scattered-light glitches from gravitational wave data, *Class. Quant. Grav.* **40**, 165005 (2023), [arXiv:2301.10491 \[gr-qc\]](#).
- [25] C. Chatterjee, K. McGowan, S. Deshmukh, N. Tyler-Howard, and K. Jani, Machine learning confirms gw231123 is a “lite” intermediate mass black hole merger, *The Astrophysical Journal Letters* **995**, L6 (2025).
- [26] R. Bondarescu, A. Lundgren, and R. Macas, Quasi-physical model for removing short glitches from LIGO and Virgo data, *Phys. Rev. D* **108**, 122004 (2023), [arXiv:2309.06594 \[gr-qc\]](#).
- [27] C.-H. Liao and F.-L. Lin, Deep generative models of gravitational waveforms via conditional autoencoder, *Phys. Rev. D* **103**, 124051 (2021), [arXiv:2101.06685 \[astro-ph.IM\]](#).
- [28] T. Eccleston and M. C. Edwards, Generative adversarial network for stellar core-collapse gravitational waves, *Phys. Rev. D* **110**, 104055 (2024).
- [29] I. J. Goodfellow *et al.*, Generative Adversarial Networks (2014), [arXiv:1406.2661 \[stat.ML\]](#).
- [30] J. Yan, A. P. Leung, and D. C. Y. Hui, On improving the performance of glitch classification for gravitational wave detection by using generative adversarial networks (2022).
- [31] J. Powell, L. Sun, K. Gereb, P. D. Lasky, and M. Dollmann, Generating transient noise artefacts in gravitational-wave detector data with generative adversarial networks, *Classical and Quantum Gravity* **40**, 035006 (2023).
- [32] M. Lopez *et al.*, Simulating transient noise bursts in ligo with generative adversarial networks (2022).
- [33] M. Lopez, V. Boudart, S. Schmidt, and S. Caudill, Simulating transient noise bursts in ligo with gengli (2022).
- [34] N. J. Cornish and T. B. Littenberg, Bayeswave: Bayesian inference for gravitational wave bursts and instrument glitches, *Class. Quant. Grav.* **32**, 135012 (2015).
- [35] J. McGinn *et al.*, Generalised gravitational wave burst generation with generative adversarial networks, *Class. Quant. Grav.* **38**, 155005 (2021).
- [36] T. Dooney *et al.*, Time-domain reconstruction of signals and glitches in gravitational wave data with deep learning, *Physical Review D* **10.1103/s91m-c2jw** (2025).
- [37] T. Dooney, R. L. Curier, D. S. Tan, M. Lopez, C. Van Den Broeck, and S. Bromuri, One flexible model for multiclass gravitational wave signal and glitch generation, *Phys. Rev. D* **110**, 022004 (2024).
- [38] T. Dooney, S. Bromuri, and L. Curier, Dvgan: Stabilize Wasserstein GAN training for time-domain gravitational wave physics, in *2022 IEEE International Conference on Big Data (Big Data)* (Osaka, Japan, 2022) pp. 5468–5477.
- [39] M. Zevin *et al.*, Gravity spy: integrating advanced LIGO detector characterization, machine learning, and citizen science, *Class. Quant. Grav.* **34**, 064003 (2017).
- [40] J. Glanzer, S. Banagiri, S. B. Coughlin, S. Soni, M. Zevin, C. P. L. Berry, O. Patane, S. Bahaadini, N. Rohani,

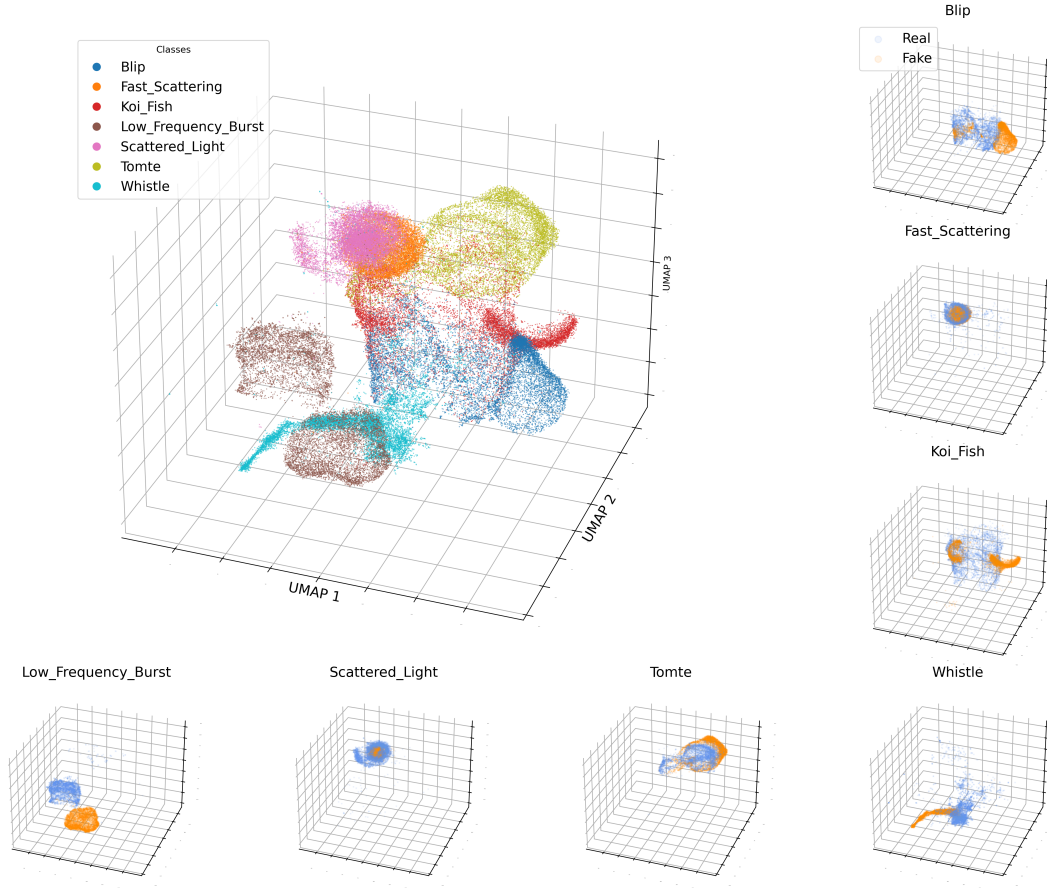


FIG. 13: Three-dimensional UMAP projection of real (orange) and DiT-generated (blue) glitches across all seven classes. The large panel shows the overall structure, while smaller panels display each class individually. Overlapping clusters indicate morphological similarity between real and synthetic samples; separable clusters for most classes indicate that DiT does not reproduce the full morphological structure of real glitches, in contrast to GlitchGAN (Figure 5).

- K. Crowston, V. Kalogera, C. Østerlund, L. Trouille, and A. Katsaggelos, Data quality up to the third observing run of advanced ligo: Gravity spy glitch classifications, *Classical and Quantum Gravity* **40**, 065004 (2023).
- [41] Y. Wu, M. Zevin, C. P. L. Berry, K. Crowston, C. Østerlund, Z. Doctor, S. Banagiri, C. B. Jackson, V. Kalogera, and A. K. Katsaggelos, Advancing glitch classification in gravity spy: Multi-view fusion with attention-based machine learning for advanced ligo’s fourth observing run (2024), [arXiv:2401.12913 \[gr-qc\]](#).
- [42] T. T. Cai and R. Ma, Theoretical foundations of t-sne for visualizing high-dimensional clustered data (2022), [arXiv:2105.07536 \[stat.ML\]](#).
- [43] L. McInnes, J. Healy, and J. Melville, UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction, arXiv preprint (2018), [arXiv:1802.03426](#).
- [44] T. A. Ferreira, G. González, and O. Salas, An analysis of ligo glitches using t-sne during the first part of the fourth ligo-virgo-kagra observing run (2025), [arXiv:2512.03440 \[astro-ph.IM\]](#).
- [45] T. A. Ferreira and G. González, Using t-sne for characterizing glitches in ligo detectors, *Classical and Quantum Gravity* **42**, 105010 (2025).
- [46] S. Deshmukh, C. Chatterjee, A. Petulante, T. A. Ferreira, and K. Jani, The sound of noise: Leveraging the inductive bias of pre-trained audio transformers for glitch identification in ligo (2026), [arXiv:2601.20034 \[astro-ph.IM\]](#).
- [47] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, Deep unsupervised learning using nonequilibrium thermodynamics (2015).
- [48] M. Cabero *et al.*, Blip glitches in Advanced LIGO data, *Class. Quant. Grav.* **36**, 15 (2019), [arXiv:1901.05093 \[physics.ins-det\]](#).
- [49] S. Soni *et al.*, Discovering features in gravitational-wave data through detector characterization, citizen science and machine learning, *Class. Quant. Grav.* **38**, 195016 (2021), [arXiv:2103.12104 \[gr-qc\]](#).
- [50] L. Nuttall *et al.*, Improving the Data Quality of Advanced LIGO Based on Early Engineering Run Results, *Class. Quant. Grav.* **32**, 245005 (2015), [arXiv:1508.07316 \[gr-qc\]](#).
- [51] M. Arjovsky, S. Chintala, and L. Bottou, Wasserstein generative adversarial networks, in *Proceedings of the 34th International Conference on Machine Learning*, Proceedings of Machine Learning Research, Vol. 70, edited by D. Precup and Y. W. Teh (PMLR, 2017) pp.

- 214–223.
- [52] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, [Improved training of wasserstein gans](#) (2017).
 - [53] M. Mirza and S. Osindero, Conditional generative adversarial nets, arXiv preprint (2014), [arXiv:1411.1784](#).
 - [54] M. Abadi *et al.*, [TensorFlow: Large-scale machine learning on heterogeneous systems](#) (2015), software available from tensorflow.org.
 - [55] J. Ho, A. Jain, and P. Abbeel, [Denoising diffusion probabilistic models](#) (2020), [arXiv:2006.11239 \[cs.LG\]](#).
 - [56] J. Ho and T. Salimans, [Classifier-free diffusion guidance](#) (2022), [arXiv:2207.12598 \[cs.LG\]](#).
 - [57] D. M. Macleod, J. S. Areeda, S. B. Coughlin, T. J. Massinger, and A. L. Urban, GWpy: A Python package for gravitational-wave astrophysics, [SoftwareX](#) **13**, [100657](#) (2021).
 - [58] E. B. Wilson, Probable inference, the law of succession, and statistical inference, [Journal of the American Statistical Association](#) **22**, 209 (1927), <https://www.tandfonline.com/doi/pdf/10.1080/01621459.1927.10502>
 - [59] J. Areeda, Are blips and koi fish related?, <https://blog.gravityspy.org/2017/10/19/are-blips-and-koi-fish-related/> (2017), accessed on 11-06-2025.
 - [60] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, CNN-generated images are surprisingly easy to spot... for now, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020) pp. 8695–8704.